

Poznan University of Technology
Faculty of Computing
Institute of Computing Science

Master's thesis

INTERPRETABLE METHODS FOR DATA-TO-TEXT GENERATION

Dominik Maćkowiak, 151915

Supervisor
Mateusz Lango, PhD

Poznań, 2026

Abstract

Data-to-text generation converts structured information into natural-language text, but direct neural generation can make content selection and realization difficult to inspect. This thesis develops an interpretable data-to-text pipeline that separates meaning-representation construction from surface realization. Raw structured instances from four domains are converted into normalized subject–predicate–object semantic triples using three LLM-based pipeline variants. A domain-specialized model trained with Quantized Low-Rank Adaptation (QLoRA) is also examined as a single-pass alternative. The resulting triples are verbalized by executable programs constructed through evolutionary search: an LLM proposes source-code mutations, while OpenEvolve evaluates and selects the resulting programs. The methods are evaluated on heterogeneous domain data and on the WebNLG RDF-to-text benchmark using reference-based, semantic, and LLM-based measures. The explicit triples, executable realizers, and evolution records provide artifacts for analyzing and reproducing the generation process.

Streszczenie

Generowanie tekstu na podstawie danych (data-to-text) polega na przekształcaniu informacji strukturalnych w tekst języka naturalnego, jednak bezpośrednie generowanie za pomocą modeli neuronowych może utrudniać analizę wyboru treści i sposobu jej realizacji. W pracy opracowano interpretowalny potok generowania, który rozdziela konstrukcję reprezentacji znaczeniowej od realizacji powierzchniowej. Surowe instancje strukturalne z czterech dziedzin są przekształcane w znormalizowane trójki podmiot–predykat–obiekt za pomocą trzech wariantów potoków opartych na dużych modelach językowych. Zbadano również model wyspecjalizowany dla danej dziedziny, dostrajany metodą QLoRA, który może tworzyć oba wyjścia w jednym przebiegu. Trójki są następnie zamieniane na tekst przez wykonywalne programy konstruowane z użyciem wyszukiwania ewolucyjnego: model językowy proponuje modyfikacje kodu, a OpenEvolve uruchamia i ocenia powstałe programy. Metody oceniono na zróżnicowanych danych dziedzinowych oraz na benchmarku WebNLG, wykorzystując metryki oparte na referencjach, ocenie semantycznej i modelach językowych. Jawne trójki, wykonywalne realizatory oraz zapis procesu ewolucji ułatwiają analizę i odtwarzanie procesu generowania.

Acknowledgements

I gratefully acknowledge the Poznan Supercomputing and Networking Center (PCSS) for providing API access to DeepSeek models used in this research. I also acknowledge PLGrid for the GPU computing resources used for experiments on the Athena cluster, and Poznan University of Technology (PUT) for access to additional GPU resources through its SLURM infrastructure.

Contents

Acknowledgements	iv
1 Introduction	1
1.1 Motivation	1
1.2 Thesis structure	3
2 Data-to-Text Generation	4
2.1 Problem definition	4
2.1.1 Data-to-Text task	4
2.1.2 RDF and semantic triples	4
2.1.3 Generation stages	5
2.1.4 Properties and challenges	6
2.2 Methods	6
2.2.1 Rule-based and template-based systems	6
2.2.2 Statistical and neural sequence-to-sequence systems	7
2.2.3 Transformers and pretrained models	7
2.3 Interpretable D2T approaches	8
2.4 Performance Evaluation	9
2.4.1 Evaluation dimensions	9
2.4.2 Reference-based automatic metrics	9
BLEU	9
METEOR	9
BERTScore	10
BLEURT	10
2.4.3 Source-aware and semantic evaluation	10
2.4.4 Human evaluation	10
2.4.5 LLM-based judging	11
3 Building interpretable meaning representations from complex data	12
3.1 Method Overview	12
3.1.1 Conversion Methods Overview	13
3.2 Data conversion with LLMs	14
3.2.1 Text-First Extraction with Normalization	14
3.2.2 Blueprint-Iterative Refinement	15
3.2.3 Evolving Catalog	17
3.3 Fine-tuned model for data conversion	18
3.3.1 Supervised Fine-Tuning and QLoRA	18
3.3.2 Training Data Construction	19

4	Surface realization — evolving programs using AlphaEvolve	21
4.1	Method Overview	21
4.2	Program construction and coding agents	22
4.3	AlphaEvolve	22
4.3.1	Evolutionary formulation	22
4.3.2	Architecture	23
4.3.3	Task specification and program representation	23
4.3.4	Prompt sampling and creative generation	24
4.3.5	Evaluation, evolutionary storage, and distributed execution	25
4.3.6	Scope and limitations of AlphaEvolve	25
4.4	OpenEvolve	25
4.4.1	Open-source implementation	25
4.4.2	Execution architecture used in the thesis	25
4.4.3	Extensions made for the experiments	26
4.5	Program database and initial program	27
4.5.1	Program records and persistence	27
4.5.2	Initial program	27
4.6	Program selection and prompt engineering	28
4.6.1	Parent and inspiration selection	28
4.6.2	Prompt composition	28
4.6.3	SEARCH/REPLACE instructions and their extensions	28
4.7	Large Language Model	29
4.7.1	Parsing the LLM response	29
4.7.2	Invalid programs and execution safety	29
4.8	Evaluation	30
4.8.1	Evaluation and feedback	30
4.8.2	Reference-based evaluation	30
4.8.3	Artifacts and continuation	30
5	Experiments	31
5.1	Meaning representation	31
5.1.1	Data Collection	31
5.1.2	Experimental setup for pipelines	32
5.1.3	Human evaluation for pipelines	34
5.1.4	Correlation between LLM and human evaluation	35
5.1.5	Experimental setup for fine-tuning	37
5.1.6	Training Configuration	37
5.1.7	Evaluation of fine-tuned models	39
5.2	Surface realization	41
5.2.1	WebNLG dataset and evaluation protocol	41
5.2.2	Evolution variants	41
5.2.3	Evolution results	42
5.2.4	Baseline systems	45
5.2.5	Three-domain comparison	46
5.3	Whole system evaluation	47
5.3.1	Data construction	47
5.3.2	Evolution and evaluation protocol	47

6	Discussion	49
6.1	Meaning representation	49
6.1.1	Pipeline comparison	49
6.1.2	Fine-tuning as pipeline distillation	50
6.2	Surface realization	50
6.2.1	Evolutionary fitness and domain effects	50
6.2.2	Comparison with baselines	51
6.3	Evaluation validity and limitations	51
6.4	End-to-end evaluation and future work	52
7	Summary	53
7.1	Contributions and findings	53
7.2	Evaluation conclusions	54
7.3	Limitations and future work	54
	Bibliography	56
A	Prompts	59
A.1	Evolution prompts for surface realization	59
A.1.1	Evolution system prompt	59
A.1.2	Evolution user prompt	60
A.1.3	Evolution-history components	61
A.1.4	Artifact message	61
A.1.5	Repair prompts	62
A.2	LLM-as-a-judge prompt	62
A.2.1	Meaning-representation pipeline prompts	62
	Summary	63
	Additions	64
	Faithfulness	64
	Omissions	64
A.2.2	Surface-realization judge prompts	65
A.2.3	Grammaticality prompt	66
A.2.4	Omissions prompt	66
A.2.5	Additions prompt	66
A.2.6	Zero-shot prompt	67
A.3	Text-First Extraction with Normalization	67
A.4	Blueprint-Iterative Refinement	68
A.5	Evolving Catalog	68
B	Prompts for Fine-tuned Model	70
C	Initial Program	71

Chapter 1

Introduction

1.1 Motivation

Data-to-text (D2T) generation concerns the automatic conversion of structured information into natural-language text. It is useful whenever information stored as records, tables, databases, time series, or knowledge graphs must be converted in a linguistic form, which is more suitable for a human reader, a conversational interface, or another language-based system. D2T systems must therefore solve two related problems: they must select and organize the information that should be communicated, and then express that information in text that is fluent, coherent, and faithful to its source [10, 21].

Recent neural and large language model based systems have substantially improved the fluency and flexibility of generated text. However, fluent text does not necessarily preserve the content of the input. A system may omit a fact, introduce an unsupported statement, select different facts on different executions in a way that changes their apparent meaning. Additionally, a direct neural generator, these intermediate choices are not usually represented as explicit objects that a developer can inspect, modify, or compare with the source. This makes it difficult to identify whether an error resulted from content selection, content ordering, aggregation, lexicalization, or surface realization. [33, 8, 7].

Before neural generation became dominant, D2T systems were commonly built using manually authored rules and templates. A rule-based system specifies conditions and operations for selecting, ordering, and combining facts, while a template provides a sentence pattern into which values can be inserted. Such systems are easy to inspect and can provide strong control over which source facts are realized. Their limitation is not that rules are inherently expensive, but that the engineering effort grows as the task becomes more varied. New predicates, combinations of facts, aggregation patterns, domains, and linguistic constructions may each require additional design, testing, and maintenance [24, 10].

This challenge is especially visible for complex structured data. A time series, for example, may require decisions about temporal ordering, aggregation, extremes, trends, and the level of detail appropriate for the output. These decisions are not caused by the presence of time alone; they arise because the system must handle many possible structures and realizations. Neural systems reduce some of this manual engineering and offer greater linguistic variety, but their learned generation behavior is harder to inspect.

One way to make this trade-off more manageable is to separate meaning representation from surface realization. A meaning representation is an intermediate, structured description of the content to be communicated. It is independent of the exact wording of the final text and can be

expressed using, for example, logical forms, semantic frames, text plans, or graphs. An explicit meaning representation makes the selected content available for inspection before it is verbalized. The resulting decomposition can be summarized as

$$d \longrightarrow T \longrightarrow y, \quad (1.1)$$

where d denotes a structured data instance, T denotes its interpretable meaning representation, and y denotes the generated text. In this context, interpretability means that the intermediate meaning representation and the procedure that produces the text are explicit artifacts that can be inspected. Meaning representation make the facts being communicated explicit and provide an intermediate object that can be inspected, normalized, and evaluated independently of the final wording. Nevertheless, constructing consistent meaning representation from complex data and producing fluent text from it remains non-trivial tasks.

Existing work has shown that LLMs can be used to construct an interpretable rule-based meaning representation to text generator. Lango and Dušek [16] use cooperating LLM agents to design and refine such a generator (a Python program), from meaning representation, which can be inspected, executed without an LLM at inference time, and corrected by a developer when necessary.

The scope of that approach begins after a meaning representation has already been provided: it addresses just the surface realization. A further limitation of a fixed sequence of agent interactions is that it does not by itself define a systematic criterion for comparing many alternative generators. Agents can propose and critique changes, but their next action is primarily guided by the current conversation.

For these reasons, the objective of this thesis is to considers the complete D2T pipeline, from heterogeneous raw structured instances through an explicit meaning representation to the final text. Separating these two stages also allows errors introduced during meaning-representation construction to be distinguished from errors introduced during surface realization. In the first stage an LLM-based pipeline is used for the meaning-representation construction. A domain-specialized model is also examined as a way of producing both the meaning representation and a text in a single inference pass. In the second stage, the surface-realization method used in this thesis places an LLM inside an evolutionary optimization loop inspired by AlphaEvolve [19], a framework for evolutionary program improvement using LLM-generated mutations and automatic evaluation. The advantage of such an approach over the agents is that it evaluates every proposed program using the same evaluator, retains high-scoring and structurally diverse candidates, and uses the accumulated results to guide subsequent modifications. This makes it possible to reject plausible but ineffective changes and to revisit alternative realization strategies rather than relying on a single refinement trajectory. Reference-based and semantic metrics are used to evaluate generation quality and faithfulness. Interpretability is supported by exposing the meaning representation and the human-readable program code.

The work makes the following contributions:

- three LLM-based pipeline designs are proposed for constructing meaning representation from heterogeneous structured data;
- a domain-specialized training procedure that distills the conversion process into a large language model;
- an evolutionary surface-realization method in which LLM-generated program mutations are executed and selected using automatic evaluation;

- a systematic empirical evaluation of the proposed methods across multiple domains using reference-based, semantic, and LLM-based measures.

1.2 Thesis structure

The structure of the thesis is as follows. Chapter 2 introduces the D2T task, reviews the principal generation approaches, discusses interpretability, and describes the evaluation methods used in this area. Chapter 3 presents the meaning-representation construction stage of the proposed pipeline. It describes how complex structured instances are converted into meaning representation using LLM-based pipelines and a domain-specialized fine-tuned model.

Chapter 4 presents the surface-realization stage, namely the conversion of semantic triples into natural-language text. It introduces the AlphaEvolve and OpenEvolve frameworks and explains how an executable program for verbalizing triples is selected, mutated, evaluated, and retained during evolution. Chapter 5 describes the experimental datasets and configurations, reports the results for meaning-representation construction and surface realization, and evaluates the complete pipeline. Chapter 6 discusses the results, limitations, and broader implications of the proposed approach. Finally, Chapter 7 summarizes the thesis and presents its conclusions. The appendices contain the prompts used by the conversion methods and the initial surface-realization program.

Chapter 2

Data-to-Text Generation

Data-to-text (D2T) generation is a branch of natural language generation in which a system converts structured input data into natural-language text [10, 21]. Inputs may be tables, database records, time series, or graphs. The output should communicate the source content fluently, coherently, and without unsupported statements. D2T is thus a mapping from a structured input representation to a linguistic output.

2.1 Problem definition

2.1.1 Data-to-Text task

The general aim of D2T generation is to produce a textual description y from structured input x :

$$G(x) = y, \tag{2.1}$$

where G is a data-to-text generator.

D2T is distinguished from text-to-text generation by the nature of its source. In machine translation, summarization, and paraphrasing, the input is already natural language. In D2T, the generator must determine what the source data means for the current communicative purpose, organize that content into a discourse, and realize the resulting plan as sentences. The required output can range from a short dialogue utterance to a multi-sentence report or a complete document [10, 21].

The generated text is normally expected to satisfy several requirements. *Faithfulness* concerns whether every expressed statement is supported by the source; incorrect values and additions are faithfulness errors. *Adequacy* concerns whether the selected content is appropriate for the communicative purpose, while *coverage* concerns omissions among facts that the task requires the text to communicate. The output should also be grammatical, fluent, coherent, and appropriately organized.

2.1.2 RDF and semantic triples

The Resource Description Framework (RDF) is a graph-based data model for representing statements about resources [5]. The basic unit of an RDF statement is a triple consisting of a subject, a predicate, and an object. The subject identifies the entity being described, the predicate denotes a relation or property, and the object identifies another entity or contains a literal value. A compact representation of a statement about a mobile phone can therefore be written as:

$$(\text{ex:PhoneModel}, \text{ex:hasBatteryCapacity}, "5000 \text{ mAh}"). \tag{2.2}$$

Here, `ex:` is a namespace prefix, `ex:PhoneModel` is the subject, `ex:hasBatteryCapacity` is the predicate, and the string "5000 mAh" is the object literal. In a complete RDF representation, resources and predicates are normally identified using globally resolvable identifiers, while literals may carry datatypes or language tags.

A collection of RDF triples forms a directed, labeled graph: subjects and resource objects are nodes, and predicates label the edges between them. The same entity can participate in many relations, and an object can simultaneously be the subject of other statements. RDF does not itself determine how the facts should be ordered or aggregated in a text [5]. It can therefore serve as a *meaning representation*: an explicit, structured representation of content that has been selected for communication. It preserves the selected facts and their relations while leaving wording, sentence structure, and discourse organization to the *surface realization*: the process of converting the meaning representation into natural language.

In this thesis, a simplified version of RDF triples are used, which can be written as

$$T = \{(s_i, p_i, o_i)\}_{i=1}^n, \quad (2.3)$$

where s_i is a subject, p_i is a predicate, and o_i is an object. From now on in the thesis, the name *semantic triples*, or just *triples* will be used to refer to this simplified representation. It has the same structural role as an RDF triple but does not require RDF namespaces, datatypes, or a complete RDF document. Here, T is the meaning representation: there must still be a process which decides how to order, aggregate, and phrase them. Several valid texts can describe the same input [10, 21].

For example, the triples (`PhoneModel`, `hasBatteryCapacity`, `5000 mAh`) and (`PhoneModel`, `hasMainCamera`, `108 MP`) can be realized as one sentence or two. Their representation alone does not prescribe either choice.

2.1.3 Generation stages

Classical descriptions of natural language generation divide D2T into a sequence of steps [24, 10]:

1. **Content determination and analysis** determines which source facts should be communicated and may derive summaries, extrema, or trends. For example, a time series can support the statement that a value is rising rather than require every observation to be listed.
2. **Document structuring** determines the order in which selected facts or entities are introduced. The ordering can follow temporal, spatial, causal, or discourse-based relations.
3. **Content aggregation** groups related facts into clauses, sentences, or paragraphs. Aggregation can reduce repetition and make the relations between facts explicit.
4. **Lexicalization** selects words and phrases that express the predicates and values in the target language.
5. **Referring expression generation** determines how entities should be mentioned, for example by using a proper name on first mention and a pronoun or a shorter description later in the discourse.
6. **Surface realization** converts the resulting plan into grammatical natural-language sentences.

The first two decisions determine “what to say?”, and aggregation determines how selected content is grouped before realization. Lexicalization, referring expression generation, and surface realization address “how to say it?”. The first group can produce a meaning representation, such as an ordered set of semantic triples or a sentence plan; the second group converts that meaning representation into text. The distinction is useful even when all stages are implemented by a single neural model because it identifies different sources of errors and possible intermediate artifacts [18].

2.1.4 Properties and challenges

There are many difficulties in D2T generation. They depend on the input representation, the number and relations of facts, the required discourse scope, and whether the task includes content determination. A fixed schema can make a template system practical, whereas varied or interconnected records increase the engineering or learning burden.

Faithfulness is a particularly important challenge for learned generators. A neural system can generate fluent text that contains an incorrect value, omits a required fact, or adds information not supported by the source. The latter two types of error are commonly studied as omissions and hallucinations [33, 8]. A rule-based system can avoid many such errors by construction, but only when its rules correctly cover the input and realization cases.

Another challenge is the multiplicity of valid realizations. A single set of triples can be expressed with different word choices, sentence boundaries, and fact orderings. A metric that rewards only agreement with one reference can therefore penalize a valid output. For learned systems, this issue affects both training and evaluation and motivates the use of multiple references, source-aware metrics, and human or model-based assessment [7, 20].

2.2 Methods

2.2.1 Rule-based and template-based systems

The earliest D2T systems were built as manually engineered pipelines. A computational linguist or system developer defined rules for selecting and ordering facts, templates for common sentence structures, and lexicalization procedures for inserting values into those templates. A template such as “⟨entity⟩ is located in ⟨place⟩” can be instantiated whenever the input contains a corresponding location relation.

The modular structure of these systems offers several advantages. The rules can be inspected directly, unsupported facts can be excluded by construction, the output style can be controlled precisely and the runtime generation is fast. These properties made templates attractive in domains such as weather forecasting, sports reporting, and dialogue systems [24, 10].

Their main limitation is the cost of domain engineering. New predicates, entities, combinations of facts, and linguistic constructions require new rules or templates. A finite template inventory also restricts lexical and syntactic variation. As the domain becomes more varied, maintaining interactions between content selection, aggregation, referring expressions, and surface realization becomes difficult. The resulting systems can be faithful and controllable while still sounding repetitive or unnatural.

2.2.2 Statistical and neural sequence-to-sequence systems

Statistical D2T systems learn generation decisions from aligned data rather than defining each decision manually. Depending on the architecture, they can model content selection, ordering, sentence planning, or surface realization. Earlier work includes probabilistic approaches to these separate decisions, learned alignments, and statistical template induction [29, 10, 21].

Compared with fully handwritten systems, statistical methods can generalize to combinations that were not explicitly covered by a template. Their behavior is nevertheless dependent on the quality, size, and domain coverage of the aligned corpus. Sparse observations or poor alignments can lead to omissions, incorrect lexicalization, or incoherent ordering.

Neural D2T systems are statistical models that formulate generation as conditional language modeling. An encoder represents the structured input, and a decoder generates the output one token at a time. Recurrent encoder–decoder models with attention were among the first widely used neural architectures for the task. At each decoding step, attention weights the encoded input records that are most relevant to the next token [10, 21].

The input may first be linearized into a token sequence, or it may be processed by a specialized encoder. Graph encoders can represent RDF nodes and labeled edges, while hierarchical table encoders can separately represent rows, fields, and cell values. Then a decoder, such as a Copy mechanisms, allow names, dates, and numbers to be copied from the input rather than generated from the vocabulary, which helps preserve rare or previously unseen values [10, 21].

Longer documents introduce additional problems. The decoder must remember which records have already been mentioned, maintain a coherent discourse order, and avoid repeating or omitting facts. The RotoWire data-to-document study showed that neural models can produce fluent text while struggling with content selection and long-range structure; in some settings, templated baselines performed better on content-related measures [33].

Neural D2T systems can be organized as a modular pipeline or as an end-to-end model. In a modular neural pipeline, separate models perform decisions such as discourse ordering, text structuring, referring expression generation, lexicalization, and realization. The output of one module is passed to the next one. In an end-to-end system, the input is encoded and directly decoded into text, with no explicitly supervised intermediate plan.

Castro Ferreira et al. [3] compared neural pipeline and end-to-end systems for RDF-to-text generation using GRU and Transformer architectures. Their results indicate that explicit intermediate steps can improve generation quality and generalization to unseen inputs. The pipeline also makes individual decisions easier to analyze, although errors can propagate between modules and the system requires intermediate annotations or procedures for constructing them.

An intermediate design is provided by Moryossef et al. [18]. Their system first creates a symbolic text plan that specifies the division of facts into sentences and their ordering, and then uses a neural model to realize that plan. The symbolic planning stage improves control over coverage and structure, while the neural realization stage supplies linguistic flexibility.

2.2.3 Transformers and pretrained models

Transformer architectures are now widely used for D2T because self-attention can model relationships among distant parts of the input and output. Modern D2T systems commonly use *pretrained Transformer language models*, whose parameters have first been optimized on a large, general collection of text, typically by predicting tokens from their context. This pretraining gives the model broad knowledge of word usage, syntax, and discourse patterns before it is applied to a particular D2T domain. It often provides a stronger basis for fluent generation than learning a model solely

from a small domain-specific corpus, although the outcome depends on the model, adaptation data, and decoding procedure [21, 12].

To adapt such a model to D2T, the structured input is serialized or encoded and paired with a target text. *Supervised fine-tuning* continues training the pretrained parameters on these input–output pairs, increasing the probability of the target text conditioned on the source data.

An alternative is prompt-based generation, in which the pretrained model is given instructions and a serialized input at inference time without updating its parameters. Prompting avoids task-specific training but leaves the desired realization behavior in the model parameters, prompt wording, and decoding choices. Both prompting and fine-tuning can generate fluent text while omitting, altering, or adding source facts, especially when the target domain differs from the pretraining or fine-tuning data. Their learned decision process is therefore less directly inspectable than an explicit rule or plan [21, 12].

2.3 Interpretable D2T approaches

Interpretability in D2T generation concerns the ability to understand why a particular text was produced from a particular input. An interpretable system should make the connection between source facts, generation decisions, and output phrases sufficiently explicit for a researcher or developer to inspect. It is related to, but distinct from, controllability, and auditability: an interpretable procedure may still make an incorrect decision, but it provides an artifact through which that decision can be examined.

Recent work uses an LLM during system construction rather than at every generation step. Warczyński, Lango, and Dušek [32] use an LLM to construct a fully interpretable rule-based D2T generator in Python. The generated source code, rather than the LLM response at runtime, contains the rule inventory and realization procedure. It can be inspected, edited, tested, and executed on a CPU independently of the construction model. This makes the construction approach directly comparable with direct prompting, where each output requires another model invocation.

The approach also has limitations. The initial program can omit a predicate, encode an unsuitable realization rule, or fail on an unseen combination of facts. Correcting such behavior requires another construction or revision step, and a single generated program does not provide a systematic record of competing program designs and their measured quality.

Lango and Dušek [16] extend this direction with a neurosymbolic RDF-to-text framework in which multiple LLM agents collaborate to construct and refine rule-based Python code. The agents receive RDF triples and produce an executable generator without in-domain human reference texts or supervised backpropagation. Their resulting program remains the inspectable artifact: it can be executed without the construction model at inference time, and a developer can inspect or modify its rules.

This multi-agent construction process reduces the need to manually author a rule base, but it still begins after a meaning representation has been supplied and therefore does not address conversion of complex raw data into a meaning representation. Its quality also depends on the agents’ prompts, generated code, and critiques. A fixed sequence of agent interactions follows one refinement trajectory; it does not by itself evaluate and retain a population of alternative program structures under a common objective.

The method proposed in Chapter 4 repeatedly mutates candidate surface-realization programs, executes them on a fixed evaluator, and retains measured alternatives. This provides basis for accepting or rejecting a program change while preserving multiple realization strategies for later exploration.

2.4 Performance Evaluation

2.4.1 Evaluation dimensions

Evaluating D2T output requires assessing both the linguistic form of a text and its relationship to the structured source. A grammatically fluent text can still omit a required fact or introduce an unsupported one. Conversely, multiple texts can be equally valid realizations of the same input, so disagreement with a single human-written reference does not necessarily indicate an error. These properties motivate an evaluation that combines complementary dimensions and procedures [10, 20].

Text quality includes grammaticality, fluency, readability, and coherence. *Faithfulness* concerns whether the claims in the output are supported by the source data. *Coverage* concerns required source facts that were not expressed, while *non-redundancy* concerns unnecessary repetitions. Adequacy is the separate question of whether the selected content suits the communicative purpose. Some applications additionally evaluate style, diversity, latency, and computational cost.

Automatic procedures can be reference-based or source-aware. Reference-based metrics compare a generated text with one or more human-written texts. They are easy to apply when a corpus contains references, but they can penalize valid paraphrases and cannot by themselves establish whether either a reference or a candidate is supported by the source. Source-aware procedures compare the output with the structured input and can target faithfulness and coverage directly.

2.4.2 Reference-based automatic metrics

BLEU

BLEU (Bilingual Evaluation Understudy) was introduced for machine translation by Papineni et al. [23]. It measures modified precision for matching unigrams, bigrams, trigrams, and four-grams between a candidate and one or more references. A brevity penalty reduces the score of candidates that obtain high precision by being much shorter than the references. A simplified corpus-level form is

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right), \quad (2.4)$$

where p_n is the modified precision for n -grams, w_n are the weights, and BP is the brevity penalty.

BLEU is computationally inexpensive, deterministic, and useful for comparing systems under the same implementation and preprocessing conditions. Its limitations are equally important. It rewards lexical overlap rather than meaning, does not directly measure factual correctness, and can assign a low score to a faithful paraphrase. It can also assign a relatively high score to a text that shares surface forms with a reference while omitting or altering source facts.

METEOR

METEOR was proposed by Banerjee and Lavie [2] to improve the relationship between automatic and human evaluation. It aligns candidate and reference unigrams using exact matches and, depending on the implementation, stem and synonym matches. Precision and recall are combined in a weighted harmonic mean, and a fragmentation penalty discourages matches that occur in a disordered fashion. METEOR can therefore recognize some lexical variation that BLEU cannot. It remains reference-dependent, however, and its behavior depends on the available stemming and synonym resources.

BERTScore

BERTScore uses contextual embeddings rather than exact token identity to compare a candidate with a reference [35]. Each token in one text is matched to semantically similar tokens in the other text using cosine similarity. Precision, recall, and F1 variants can be reported. Because contextual embeddings represent related words and paraphrases more flexibly, BERTScore can capture similarities missed by n-gram metrics. It is nevertheless dependent on the pretrained model, tokenizer, language, and reference text, and semantic similarity does not guarantee factual faithfulness to the source data.

BLEURT

BLEURT is a learned evaluation metric trained to predict human judgments of generated text [25]. It combines pretrained language-model representations with further training on synthetic perturbations and human ratings. Unlike a direct n-gram count, BLEURT can learn that some substitutions, omissions, or fluency problems affect quality differently. This flexibility can improve correlation with human judgments, but it also introduces dependence on the metric model and its training distribution. Under domain shift, a metric trained or tuned on one distribution may no longer rank outputs reliably in a new domain. Model bias and the use of a reference remain relevant limitations.

2.4.3 Source-aware and semantic evaluation

Reference-based metrics treat the reference as the principal target. In D2T, the structured source should also be treated as an evaluation object. This is especially important when a reference contains information that is not present in the input or when a candidate expresses an input fact using a different wording.

PARENT (Precision And Recall of Entailed N-grams from the Table) addresses this problem for table-to-text generation [7]. It compares the candidate with both the reference and the source table. Its precision and recall calculations use entailment estimates to reward information supported by the table and to reduce the effect of divergent reference text. PARENT illustrates the value of task-specific metrics, although its original formulation is designed for semi-structured tables and requires a suitable source-to-text entailment procedure.

Dušek and Kasner [8] proposed an NLI-based semantic-accuracy metric for D2T. The structured input is converted into simple textual statements, after which natural-language inference is used in both directions. Testing whether the input entails the output helps detect unsupported information, while testing whether the output entails each input fact helps detect omissions. The resulting labels can distinguish faithful outputs, omissions, hallucinations, and combinations of these errors. This approach directly targets the semantic requirements of D2T, but its accuracy depends on the data-to-text conversion, the NLI model, and the assumptions about whether all input facts should be realized.

2.4.4 Human evaluation

Human evaluation remains necessary because automatic scores are proxies for quality. Annotators can assess the generated text directly or judge it together with the input representation, reference, or both. Typical criteria include grammaticality, fluency, coherence, informativeness, coverage, repetition, and factual consistency. For D2T, semantic criteria are often operationalized through

questions such as whether required facts are present and whether unsupported facts have been added. Text quality can also be measured on a Likert scale [10, 20].

Human evaluation is expensive and can be affected by annotator expertise, instructions, sample selection, and the number of annotators per item. Studies with multiple annotators should report inter-annotator agreement and describe the assumptions behind the selected statistic. The survey by Artstein and Poesio [1] discusses the use of agreement coefficients in computational linguistics. Cohen’s kappa measures chance-corrected agreement between two annotators assigning nominal categories, whereas Krippendorff’s alpha is designed to accommodate multiple annotators, missing annotations, and different measurement levels [4, 14]. A single-rater study cannot estimate agreement and should be presented as an exploratory assessment; the manual pipeline assessment in Chapter 5 has this limitation. A reliable evaluation should specify the criteria, scale, annotation procedure, number of annotators, agreement measure where applicable, and aggregation method. Human assessment is particularly important when automatic metrics disagree or when the task permits many valid verbalizations [31, 34].

2.4.5 LLM-based judging

An LLM-based judge is an automatic evaluator that leverages the capabilities of large language models. It receives a generated text, the information needed to assess it, and an explicit evaluation criteria. The prompt asks the judge to compare the candidate output with the source, identify relevant errors, and return a rating, which can be a score, a label, or a structured explanation. The judge can be prompted to provide a rationale for its rating, which can help interpret the evaluation and identify specific issues in the output.

LLM judging complements reference-based, source-aware, and human evaluation. It can reward a faithful text and can give a direct explanations of the errors.

Chapter 3

Building interpretable meaning representations from complex data

3.1 Method Overview

The central problem addressed in this work is the conversion of complex structured data into interpretable meaning representations. Modern data sources frequently provide information in structured formats such as JSON or XML, containing nested fields, time series, categorical values, and hierarchical relationships. While these formats are efficient for data storage and exchange, they are not directly amenable to semantic reasoning, or natural language understanding tasks. The challenge lies in extracting the semantic content from these structures and representing it in a standardized, machine-interpretable form.

The goal is to transform structured data into semantic triples—subject-predicate-object statements that capture the essential meaning of the data. For example, consider a JSON object describing a mobile phone with nested specifications:

```
{"name": "PhoneModel", "display": {"size": "6.5 inch", "resolution": "2400x1080"},  
  "battery": {"capacity": "5000 mAh"}, "camera": {"main": "108 MP"}}
```

This structured data should be converted into triples such as:

- (PhoneModel, hasDisplaySize, 6.5 inch)
- (PhoneModel, hasDisplayResolution, 2400x1080)
- (PhoneModel, hasBatteryCapacity, 5000 mAh)
- (PhoneModel, hasMainCamera, 108 MP)

These triples capture the semantic relationships in a format that can be directly used for querying, and reasoning.

This transformation presents several challenges that must be addressed by any practical method. First, the input data varies significantly in structure and complexity: some sources contain flat key-value pairs, while others have deeply nested hierarchies, time series with dozens of data points, or categorical attributes with multiple values. Second, the extraction must preserve semantic accuracy, ensuring that the generated triples faithfully represent the information in the source data without introducing errors or omissions. Third, predicate normalization is essential for consistency: the same relationship should be expressed using the same predicate across different instances (e.g., always `hasBatteryCapacity` rather than sometimes `battery_capacity`, sometimes `has_battery`,

etc.). Fourth, the methods must scale to handle large datasets efficiently, as real-world applications may involve thousands or millions of instances.

To address these challenges, two complementary approaches are investigated in this work. The first consists of pipeline-based methods that use large language models in a multi-stage process: first generating natural-language references from the structured data, then extracting semantic triples from those references. The second approach distills this behavior into a single domain-specialized fine-tuned model that produces both the natural-language text and the semantic triples in a single pass. Both approaches are described in detail in the following sections.

3.1.1 Conversion Methods Overview

Two complementary approaches are investigated for converting raw structured instances into semantic triples. The first consists of three pipeline variants that use a general-purpose LLM to first generate reference texts and then extract triples, while the second distills this behavior into a single domain-specialized fine-tuned model.

Pipeline-based methods. The first approach, described in Section 3.2, consists of three pipeline variants that convert data to triples using large language models. All three methods follow a common two-stage structure: in the first stage, each raw structured instance is converted into a natural-language reference text; in the second stage, semantic triples are extracted from that text. The three variants differ in how they manage predicate consistency and validation.

The fundamental challenge is that predicate consistency—ensuring the same semantic relationship is expressed with the same predicate across all instances—requires some form of coordination between extraction calls, but such coordination introduces sequential dependencies that reduce throughput. A fully parallel approach maximizes speed but produces inconsistent predicates; a fully sequential approach ensures consistency but is slow. The three variants explore different points in this tradeoff space:

- **Text-First Extraction with Normalization** prioritizes throughput by extracting triples independently in parallel, accepting predicate inconsistency during extraction and resolving it through a post-hoc normalization stage. This approach is the fastest but requires additional LLM calls for normalization.
- **Blueprint-Iterative Refinement** prioritizes interpretability and control by creating an explicit, human-inspectable specification (the blueprint) that defines the predicate vocabulary upfront. The blueprint is iteratively validated and refined against reference texts before extraction. This approach provides the most structured and auditable output but requires multiple validation rounds.
- **Evolving Catalog** seeks a middle ground by building predicate consistency incrementally during a sequential bootstrap phase, then switching to batch processing once the predicate vocabulary stabilizes. This hybrid approach balances consistency and throughput, adapting to the data’s complexity.

Fine-tuned model. The second approach, described in Section 3.3, distills the behavior of the multi-stage pipeline into a single domain-specialized model. Using *QLoRA* (Quantized Low-Rank Adaptation), a model is fine-tuned to produce both natural-language text and semantic triples in a single decode pass. This approach trades the flexibility of the pipeline for reduced inference

latency and improved consistency, at the cost of requiring per-domain fine-tuning on reference outputs produced by the pipeline.

3.2 Data conversion with LLMs

Pipeline overview. All three methods follow a common two-stage structure. In the first stage, each raw structured instance is converted into a natural-language reference text. In the second stage, semantic triples are extracted from that text. This text-first approach decouples verbalization from triple extraction, allowing each stage to be optimized independently and batched for throughput. The rationale for this decomposition is that natural language serves as an intermediate representation that generalizes the input data, while the structured triple output can be directly used for downstream tasks such as program synthesis.

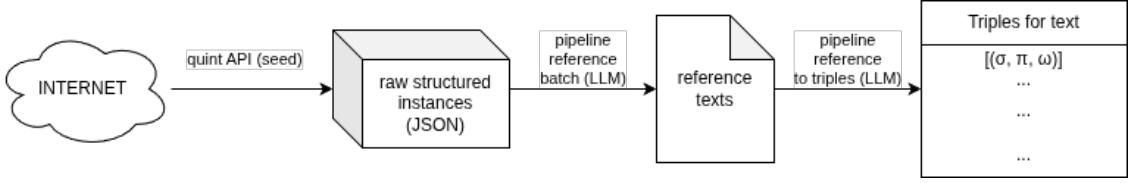


FIGURE 3.1: General pipeline flow: raw structured data from quintd is first converted to reference texts, then to semantic triples using one of the three pipeline methods.

3.2.1 Text-First Extraction with Normalization

Text-First Extraction with Normalization decomposes the data-to-triples problem into two fully batched LLM stages and a normalization stage. The method deliberately avoids any inter-instance coupling during extraction: each reference and each triple set is produced independently and in parallel, which makes the method stateless, at the cost of not enforcing predicate consistency during extraction.

Stage 1 — Reference text generation. In the first stage the model is invoked once per instance under a `text-generation system prompt`, P_{txt} (see Appendix A.3), whose objective is to convert one structured data instance into concise natural language and to return the result as a JSON object of schema `{"text": "..."}1. The prompt further instructs the model to capture the most important information, trends, extremes, and notable conditions, and to summarize time series (e.g., weather forecasts) at a high level rather than listing every data point.`

All N requests are packaged as a single OpenAI Batch API job and a single request outputs t_i , where t_i is the generated reference text for instance d_i . This batched stage yields, for every instance, a natural-language reference t_i that constitutes the intermediate representation from which triples are subsequently derived.

Stage 2 — Triples-from-text extraction. A second OpenAI Batch is then assembled in which every reference t_i is paired with a `triples-from-text system prompt`, P_{tr} (see Appendix A.3), whose objective is to extract semantic triples from natural language text, returning JSON of schema `{"triples": [{"subject": "...", "predicate": "...", "object": "..."}]}`. The prompt imposes two constraints: it instructs the model to preserve the full meaning of the input text and to keep predicates in `lower_snake_case` when possible, and to avoid duplicate triples.

¹Throughout this work, JSON schemas are presented in simplified notation with placeholder values (e.g., `"text": "..."`) for readability, rather than as formal JSON Schema definitions.

Each instance is therefore an independent text-to-triples problem, and the batch can be fully parallelized. The output of a single request is T_i , where T_i is a set of triples that describe the reference text t_i .

Stage 3 — Post-hoc predicate normalization. Because Stage 2 imposes no inter-instance vocabulary control, this stage invokes a predicate-normalization routine. The routine operates on the flattened list of triples and iteratively merges semantically equivalent predicates:

Let Π be the set of distinct predicates present in the extracted triples. A Union-Find structure over Π is initialized with every predicate in its own singleton class. A *Union-Find* (disjoint-set) is a data structure that maintains a collection of disjoint sets and supports two operations: **find**(x) returning the representative element of the set containing x , and **union**(a , b) merging the sets containing a and b . A *singleton class* is an equivalence class containing only a single element.

For each round $r = 1, 2, \dots$ (bounded by $2|\Pi|$):

- The model is prompted with the current set of canonical predicates (class representatives) and asked to propose candidate equivalent pairs under a **candidate-pairs system prompt** (see Appendix A.3).
- If no candidate pairs are returned, or if the candidate set is identical to one proposed in an earlier round (indicating the model has converged or is cycling without progress), the loop terminates.
- Otherwise, every candidate pair is sent to the model under a **strict-equivalence system prompt** (see Appendix A.3) that returns a JSON object with three fields: **equivalent** (boolean), **confidence** (a score between 0 and 1), and **reason** (a textual explanation). The model is instructed to judge equivalence strictly (mergeable in a knowledge graph), not mere relatedness.
- Pairs judged equivalent are merged in the Union-Find structure. If a round produces no merges, the loop terminates.

After termination, each predicate is mapped to the canonical representative of its class, defined as the first member of the class (deterministic, order-based choice). The triples are then rewritten with these canonical predicates.

In the normalization step, the final output combines: (i) generated references, (ii) extracted triples, and (iii) normalized triples alongside the canonical mapping and the comparison audit trail.

3.2.2 Blueprint-Iterative Refinement

Blueprint-Iterative Refinement introduces an explicit notion of extraction blueprints that act as an intermediate, human-inspectable artifact between the domain declaration and the final triple extraction. An *extraction blueprint* is a structured specification containing predicate labels, example triples, example text, and transformation guidelines. The blueprints are generated top-down from the domain name, then iteratively validated and revised against the generated references until it is confirmed that they can extract triples for every instance. Only then is a single batched triple-extraction run executed under the finalized blueprint.

Stage 1 — Initial blueprint generation. Given the user-supplied problem domain string δ (e.g., weather forecast), the model is invoked once under a **blueprint-creation system prompt**, $P_{\text{blueprint}}$ (see Appendix A.4), and asked to design an extraction blueprint for a domain. Its

response is constrained by a strict JSON schema to contain exactly four fields:

$$R = (\Pi_R, E, T_{\text{ex}}, G),$$

where Π_R is a list of predicate labels, E is a list of example triples $\{(\sigma_k, \pi_k, \omega_k)\}$ using those predicates, T_{ex} is a short natural-language passage consistent with the example triples, and G is a list of free-text transformation guidelines. The response is sanitized, which deduplicates predicates and guidelines, discards malformed example triples, and falls back to the corresponding field values from the previous blueprint R_{r-1} for any field that is empty or malformed (for the initial blueprint R_0 , empty fields default to empty lists or strings). The resulting R_0 constitutes the initial blueprint. Conceptually, R_0 is a self-contained specification of how the model believes instances of the domain should be verbalized and triplelized, complete with concrete formatting conventions embedded in the example triples.

Stage 2 — Reference text generation. The reference-generation batch is identical to that of the previous method: every instance d_i is sent to the **text-generation system prompt** P_{txt} within a single OpenAI Batch, producing references t_i .

Stage 3 — Blueprint validation and iterative revision. The core of the method is an iterative loop, bounded by R_{max} . The loop maintains a current blueprint R and operates as follows for each round $r = 1, \dots, R_{\text{max}}$:

1. **Feasibility scan.** A random reference text with replacement t_i is selected and the model is invoked under a **blueprint-feasibility system prompt**, P_{feas} (see Appendix A.4), which receives the current blueprint R and the text t_i , and returns a JSON object with three fields: **possible** (boolean indicating whether the blueprint is sufficient), **reason** (textual explanation), and **blueprint_gaps** (a list of descriptions of what is missing from the blueprint in order to extract triples from t_i).
2. **Pass condition.** If the scan reaches the R_{max} without encountering a **possible = false** judgment, the blueprints are declared validated against all texts and the loop terminates.
3. **Failure and revision.** As soon as a reference t_{i^*} is judged infeasible under R , the scan halts. The **blueprint-revision system prompt**, P_{rev} (see Appendix A.4), is then invoked with the domain δ , the current blueprint R , the failing text t_{i^*} , the failure reason, and the enumerated **blueprint_gaps**. Its response is again constrained to the same four-field schema structure $R = (\Pi_R, E, T_{\text{ex}}, G)$ described in Stage 1. The model is instructed to keep existing high-quality blueprint components, but add or adjust what is necessary to cover the failing case. The sanitized result replaces R as the new current blueprint.
4. **Restart.** Because revising the blueprint may break previously validated instances or expose new gaps, the scan is restarted and the round number r set to 1. The loop thus monotonically accumulates coverage, but backtracks the scan position whenever the blueprint is modified.

If the budget R_{max} is exhausted before all texts are validated, the method emits a warning and proceeds to final extraction with the best-effort blueprint; the output records that validation was incomplete.

This stage implements a generate-test-revise loop in which the blueprint is the artifact being refined, the reference texts act as test cases, and the feasibility prompt acts as a model-internal test oracle.

Stage 4 — Final batched extraction under finalized blueprint. Once the validation loop has terminated, a single OpenAI Batch is submitted in which every reference t_i is paired

with a **triples-from-text-with-blueprint system prompt**, $P_{\text{tr}\&R}$ (see Appendix A.4). This prompt instructs the model to extract semantic triples from the natural language text using the provided blueprint, to follow the blueprint guidelines, to mimic the formatting style of the example triples, and to prefer predicates from the blueprint predicate list whenever possible. The blueprint R is serialized and injected in each request. The batch output is parsed into T_i .

3.2.3 Evolving Catalog

Evolving Catalog is a hybrid text-first data-to-triples pipeline that combines an evolving predicate catalog with a stability-triggered transition from sequential to batch processing. It is designed to keep the predicate vocabulary consistent across instances while reducing the latency overhead of per-instance, order-dependent LLM calls.

Stage 1 — Reference text generation. An OpenAI Batch is assembled in which every instance d_i is paired with the **text-generation system prompt** P_{txt} (see Appendix A.5), whose purpose is to produce a concise natural-language summary that captures the most important information, trends, extremes, and notable conditions, and summarizes time series at a high level. The batch output is parsed into text reference t_i .

Stage 2 — Sequential catalog-driven triple extraction. The second stage maintains a mutable predicate catalog C , initialized as empty. The catalog is a set of entries $\{(\pi, e_\pi)\}$, where each entry associates a predicate label π with an example triple $e_\pi = (s, \pi, o)$ in which that predicate was first observed. The catalog thus carries both a normalized vocabulary and a small set of usage examples that serve as in-context guidance for the extraction model.

The instances are processed strictly in order:

1. **Bootstrap instance** ($i = 0$). Because C is empty, the reference t_0 is sent to the model under a **first-instance system prompt**, P_{first} (see Appendix A.5), that simply asks for semantic triples from the text with concise **lower_snake_case** predicates and no duplicates. This yields a set of triples T_0 .
2. **Catalog-guided instances** ($i \geq 1$). For each subsequent instance d_i , the reference t_i is sent under a **catalog-guided system prompt**, P_{cat} (see Appendix A.5), in which the current catalog C is serialized and injected into the user prompt. The model is instructed to use only predicates from the provided catalog when possible and to introduce a new predicate only when none of the catalog predicates can describe the fact. This yields triples T_i .

After each instance, every triple in T_i is inspected. If a triple contains a predicate $\pi \notin C$, a new entry (π, e_π) is added to C , where e_π is set to that triple (the first occurrence).

Stage 3 — Stable-window heuristic and batched tail processing. A counter w tracks the number of consecutive instances that did not introduce any new predicate; it is reset to zero whenever a new predicate is added. Let W denote the stable window parameter. The sequential phase terminates as soon as $w \geq W$ and at least one instance remains unprocessed. Let i^* be the index at which this condition is first met; the catalog C at that point is declared frozen.

All instances $d_{i^*+1}, \dots, d_N$ (the tail) are then processed in a single second OpenAI Batch, again under P_{cat} and with the frozen catalog C injected into every request. Two design choices are important here:

- The tail is parallelized, hence the cost of the remaining $N - i^* - 1$ LLM calls is amortized into one batch.

- Predicates that first appear within the tail are not added back to C . The catalog is finalized at the moment of stability, which guarantees that the predicate vocabulary presented to a human evaluator is fully determined by the sequential phase and does not silently grow during batch processing.

3.3 Fine-tuned model for data conversion

The methods described in Section 3.2 rely on repeated invocations of a general-purpose large language model at inference time: each instance requires one or more LLM calls to produce reference text, extract triples, and, in some pipelines, to validate or revise intermediate artifacts. While this design keeps the pipeline flexible, it incurs substantial latency and computational cost, particularly when the same domain is processed repeatedly. An alternative is to *distill* the behaviour of the multi-stage pipeline into a single, domain-specialized model that produces both the natural-language reference and the corresponding triple set in one decode pass. This section describes how such a model is obtained via supervised fine-tuning with QLoRA.

3.3.1 Supervised Fine-Tuning and QLoRA

Supervised fine-tuning (SFT) adapts a pre-trained language model to a specific task by training it on input–output pairs. Given a pre-trained model M_θ with parameters θ and a dataset $\{(x_i, y_i)\}_{i=1}^n$ of input–output examples, SFT minimizes the negative log-likelihood of the targets [22]:

$$\mathcal{L}(\theta) = -\frac{1}{n} \sum_{i=1}^n \sum_{t=1}^{|y_i|} \log P_\theta(y_{i,t} \mid x_i, y_{i,<t}). \quad (3.1)$$

For small models (up to a few billion parameters), full SFT—updating all parameters θ —is feasible on modern GPU hardware. However, for models with tens of billions of parameters, full fine-tuning becomes prohibitively expensive: a 31-billion-parameter model in 16-bit precision requires approximately 62 GB of memory for the weights alone, and the standard Adam optimizer [13] maintains two additional moment estimates per parameter, roughly doubling the memory footprint.

Parameter-efficient fine-tuning (PEFT) methods address this by keeping the base model weights frozen and introducing a small number of trainable parameters [17]. Among these, *Low-Rank Adaptation* (LoRA) [11] has become the most widely adopted. LoRA decomposes the weight update into a low-rank product: for each frozen weight matrix W_0 , the forward pass is modified as

$$h = W_0x + \Delta Wx = W_0x + \frac{\alpha}{r}BAx, \quad (3.2)$$

In this formulation, W_0x represents the output of the original pretrained model, while ΔWx is the learned task-specific adaptation. The key insight from Hu et al. [11] is that the weight updates ΔW during fine-tuning also have low “intrinsic rank,” meaning they can be approximated by a low-rank decomposition. Rather than updating the full weight matrix W_0 , LoRA keeps W_0 frozen and learns the low-rank matrices A and B , where $\Delta W = \frac{\alpha}{r}BA$. The scaling factor $\frac{\alpha}{r}$ controls the magnitude of the adaptation relative to the pretrained weights. This decomposition reduces the number of trainable parameters from $d \times k$ (the full weight matrix) to $r \times (d + k)$ (the two low-rank matrices), where $r \ll \min(d, k)$.

where:

- $W_0 \in \mathbb{R}^{d \times k}$ is the frozen pretrained weight matrix
- d is the output dimension and k is the input dimension of the layer

- $x \in \mathbb{R}^k$ is the input vector to the layer
- $h \in \mathbb{R}^d$ is the output (hidden state) of the layer
- $A \in \mathbb{R}^{r \times k}$ is the first trainable low-rank matrix (down-projection)
- $B \in \mathbb{R}^{d \times r}$ is the second trainable low-rank matrix (up-projection)
- $r \ll \min(d, k)$ is the rank of the decomposition
- α is a scaling factor that controls the magnitude of the low-rank update

Only A and B are updated by the optimizer; W_0 remains frozen and requires no gradient storage or optimizer state. For a typical transformer with attention projections W_q, W_k, W_v, W_o and feed-forward projections $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$, LoRA adapters are attached to each projection, yielding a trainable parameter count that is a small fraction of the base model (on the order of tens of millions versus tens of billions).

At the data scales typical for domain-specific data-to-text tasks (on the order of several hundred to a few thousand labeled examples), LoRA’s low-rank constraint also acts as a regularizer: by limiting the number of trainable parameters, it reduces the risk of overfitting and preserves the general linguistic capabilities acquired during pre-training [11].

QLoRA [6] extends LoRA with two memory-reduction techniques that together allow fine-tuning a 31-billion-parameter model on a single GPU with 40 GB of VRAM. First, the frozen base model weights are quantized to *4-bit NormalFloat* (NF4), a quantization datatype that is information-theoretically optimal for normally distributed weights. A *double quantization* step further quantizes the quantization constants themselves, reducing the per-parameter overhead. Second, *paged optimizers* leverage NVIDIA unified memory to transparently page optimizer state between GPU and CPU memory when the GPU reaches capacity, avoiding out-of-memory failures during gradient accumulation steps. The forward and backward passes are performed in `bfloat16` precision: the 4-bit weights are dequantized on the fly for each matrix multiplication, preserving numerical quality close to a full-precision fine-tune while keeping the memory footprint low.

3.3.2 Training Data Construction

Rather than collecting human-annotated data-triple pairs, the training data for the fine-tuned model is produced by *distilling* one of the pipeline described in Section 3.2. For each raw structured instance d_i processed by a tripler method, the pipeline already produces a reference text t_i and a set of normalized triples $T_i = \{(s_k, \pi_k, o_k)\}$. These two outputs, aligned by instance identifier, constitute exactly the joint target that the fine-tuned model should learn to produce in a single pass.

Each training example is formatted as a three-message conversation following the base model’s chat template:

- **System prompt.** An instruction directing the model to convert one structured data instance into concise natural language and a corresponding set of RDF semantic triples (see Appendix B for the complete prompt text), and to return the result as a JSON object of schema `{"text": "...", "triples": [{"subject": "...", "predicate": "...", "object": "..."}]}`.
- **User message.** A prompt that presents the serialized instance and requests the reference text and triples (see Appendix B for the complete template). The wording and instance

serialization are kept byte-identical to the prompts used by the tripler pipeline at inference time, ensuring *prompt parity*: the token sequence seen during training matches the token sequence the model will encounter when served.

- **Assistant message (target).** A JSON string encoding the reference text and the normalized triples (see Appendix B for the expected output format): `{"text":  $t_i$ , "triples":  $T_i$ }`.

The messages are rendered through the base tokenizer’s `chat_template` function, which applies the special tokens and formatting expected by the model architecture. This ensures that the token sequences fed to the trainer are identical to those assembled by the serving engine at inference time.

A notable consequence of this distillation approach is that the fine-tuned model is bounded by the quality of the reference labels: it can learn to reproduce the pipeline’s outputs more efficiently and consistently, but it cannot exceed their quality. Improving the reference labels is an upstream concern addressed by the tripler pipeline design, not by fine-tuning.

The dataset is split into a training, evaluation and test set. The split is deterministic, controlled by a fixed random seed, so that results are reproducible across runs. One fine-tuned model is produced per domain, keeping the predicate vocabulary and verbalization style specialized to that domain.

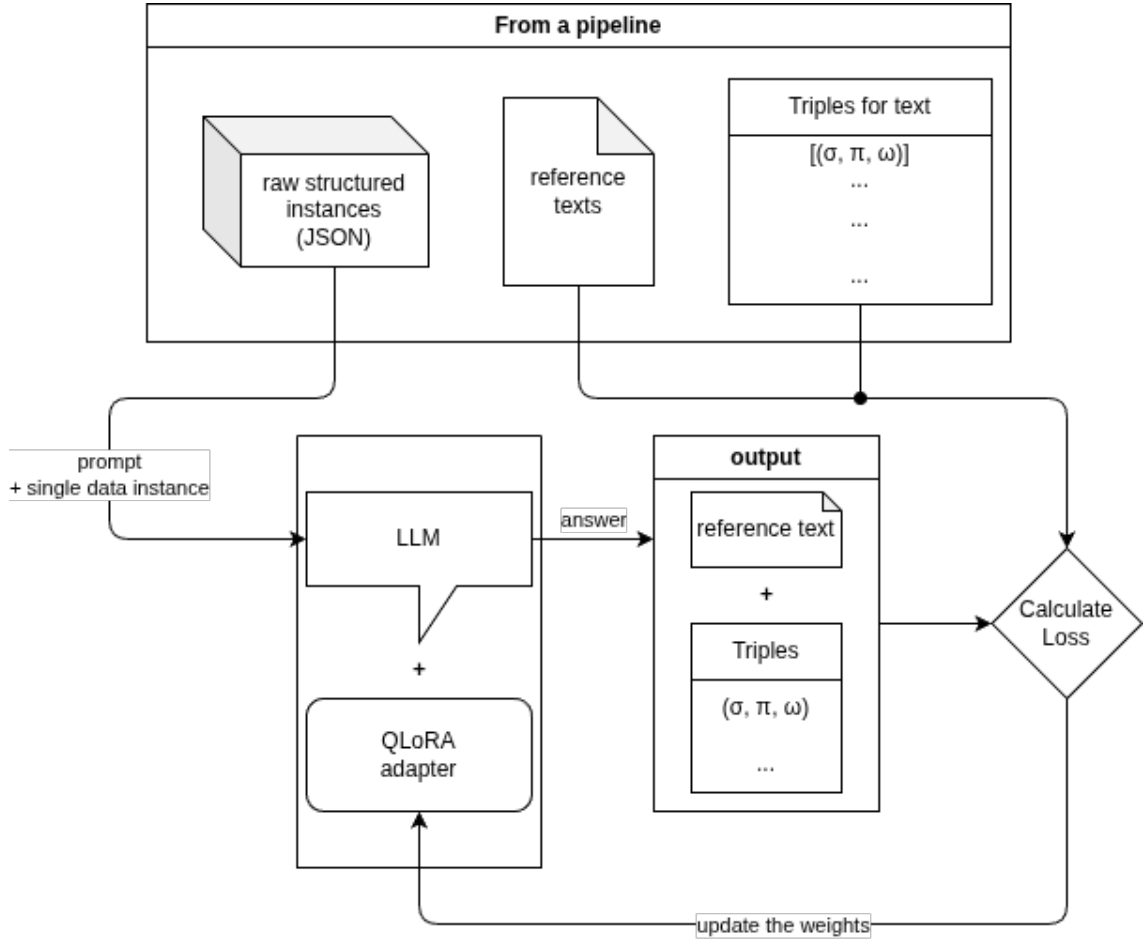


FIGURE 3.2: Fine-tuning flow: the raw structured instances, reference texts and triples are used to train a domain-specialized model via QLoRA. The resulting model can produce both outputs in a single decode pass.

Chapter 4

Surface realization — evolving programs using AlphaEvolve

4.1 Method Overview

The previous chapter considered the problem of meaning representation construction. Complex structured data was converted into semantic triples so that the information could be expressed in a compact, standardized, and machine-interpretable form. The problem considered in this chapter is the second stage of the data-to-text conversion: given the information contained in a set of triples, how should it be written in natural language?

This stage is called *surface realization*. Let $T = \{(s_i, p_i, o_i)\}_{i=1}^n$ denote a set of subject–predicate–object triples and let y denote a natural-language text. A surface realizer is a mapping

$$G : T \longrightarrow y, \tag{4.1}$$

which must express the facts in T fluently and coherently. The mapping is not limited to replacing predicates with fixed phrases. It can require entity ordering, aggregation of related facts, selection of referring expressions, realization of numerical values, and the choice of a suitable sentence structure. The generated text should preserve the information in the triples while avoiding unsupported statements and unnecessary repetition.

An LLM can perform this mapping directly for every input through a zero-shot or few-shot prompt. That approach can produce fluent text, but the realization decision is then made anew for each input. This thesis instead uses the LLM to propose changes to a Python surface-realization program. The resulting program is a reusable artifact: after an evolutionary run, it can be inspected and executed on previously unseen triple sets without another LLM call.

This idea is directly motivated by the work of Lango and Dusek [16], who describe a neurosymbolic framework in which LLM agents implement an NLG system from scratch. The generated code provides a transparent representation of the realization process and can generate text using ordinary CPU execution. The work demonstrates that LLMs can be used to build an NLG system rather than only to act as the NLG system at inference time.

The present work shares the goal of an inspectable generator, but obtains it differently. Here, Google’s AlphaEvolve idea is used instead [19]: rather than assigning several agents the task of designing and refining a program, it starts from a working baseline and searches over LLM-proposed program mutations. Each candidate is executed and selected using a task-specific objective. Unlike the reference-free construction setting of Lango and Dušek, this search uses reference lexicalizations, to evaluate candidate programs. The method is therefore an LLM-guided evolutionary

algorithm in which an LLM mutates a program to complete a general software-engineering task. Its loop is as follows:

1. select a previously evaluated parent program;
2. ask a selected LLM to propose a mutation of that program;
3. execute the mutated program on evaluation inputs;
4. calculate a scalar selection score or a diagnostic feedback; and
5. register the candidate so that it can influence later selection.

The remainder of the chapter introduces the related programming paradigms, defines AlphaEvolve’s evolutionary formulation, and then describes the OpenEvolve implementation and its evaluation modes.

4.2 Program construction and coding agents

Program synthesis searches for a program consistent with a formal specification or a set of input–output examples. Code completion, in contrast, predicts a local continuation of an existing source file. Neither description by itself implies an iterative development process. *Agentic programming* denotes a different setting in which an LLM-based system maintains a control loop: it observes a workspace, invokes tools, interprets their results, and chooses later actions while pursuing a software-development task [30].

The distinction matters here because the later described AlphaEvolve uses some ideas common in coding-agent systems—a language model, source code, execution feedback, and persistent state—without requiring a general tool-using agent. It has a restricted interface: the LLM proposes a program mutation and an evaluator returns its result. This restricted loop makes candidate comparison explicit and repeatable, which is useful when the desired artifact is a compact surface realizer rather than a complete software project.

Coding agents nevertheless motivate the use of execution feedback. Generated source can contain invalid assumptions or fail a test. The relevant question for this chapter is consequently not whether an LLM can describe a realization strategy, but whether the resulting program improves an observable realization objective. AlphaEvolve provides the selection mechanism for this question.

4.3 AlphaEvolve

4.3.1 Evolutionary formulation

AlphaEvolve is an LLM-guided evolutionary coding system for scientific and algorithmic problems whose candidate solutions can be represented as programs and evaluated automatically [19]. Let $\mathcal{P}$ denote programs satisfying a task interface and let

$$h : \mathcal{P} \longrightarrow \mathbb{R}^m \tag{4.2}$$

be an evaluator that returns one or more metrics. A designated scalar derived from these metrics is the fitness used for selection. Beginning with a runnable baseline p_0 , the system repeatedly selects a parent p , obtains an LLM-proposed mutation p' , evaluates $h(p')$, and stores the result. The stored programs then form the pool from which later parents are sampled.

The LLM generation is stochastic, but a generated program is an ordinary deterministic source artifact until it is changed again. The LLM replaces a hand-designed mutation operator by proposing source-level modifications that may express algorithmic ideas. Selection is still based on the evaluator, rather than on the persuasiveness of the LLM’s explanation.

4.3.2 Architecture

The description of AlphaEvolve separates the process of creating a program into the following cooperating components [19]:

1. The *task specification* supplies the initial program, the evaluator, optional configuration, and any fixed domain knowledge.
2. The *prompt sampler* selects previously evaluated programs and assembles the context shown to the language model.
3. The *LLM ensemble* asks a selected LLM to propose a mutation of that program. A routing or sampling policy selects one model for a mutation request; that model independently returns a candidate patch. Faster models can be sampled more often to explore more candidates, while a more capable model can be selected for less frequent changes
4. The *mutation layer* parses the model response and applies the proposed changes to the current source code. For large programs, targeted diff blocks avoid sending or rewriting unrelated code.
5. The *evaluators* execute the candidate and return its metrics. Evaluation can include cascades, parallel test cases, multiple objectives, and additional LLM-generated feedback.
6. The *program database* stores candidates, metrics and controls the balance between exploiting high-scoring programs and exploring different program structures.
7. The *distributed controller* coordinates LLM requests and evaluations asynchronously so that slow evaluations do not prevent other candidates from being processed.

The interaction between these components is deliberately cyclic. The program scores, code, and observed failures become part of the context used for subsequent mutations. The database consequently acts as an external memory for the LLM-guided evolutionary algorithm agent. [19]. The following sections describe the components in more detail.

4.3.3 Task specification and program representation

An AlphaEvolve task has three essential inputs. First, an initial program must be complete enough to run, even if its implementation is deliberately simple (the complete initial program is reproduced in Appendix C). Second, the part of the source code that may be changed is marked with `# EVOLVE-BLOCK-START` and `# EVOLVE-BLOCK-END`. The remaining code forms a fixed skeleton that defines imports, data types, and the public interface. Third, an evaluator function must be provided. The evaluator receives the generated program, executes it on the task-specific inputs, and returns a dictionary of scalar metrics.

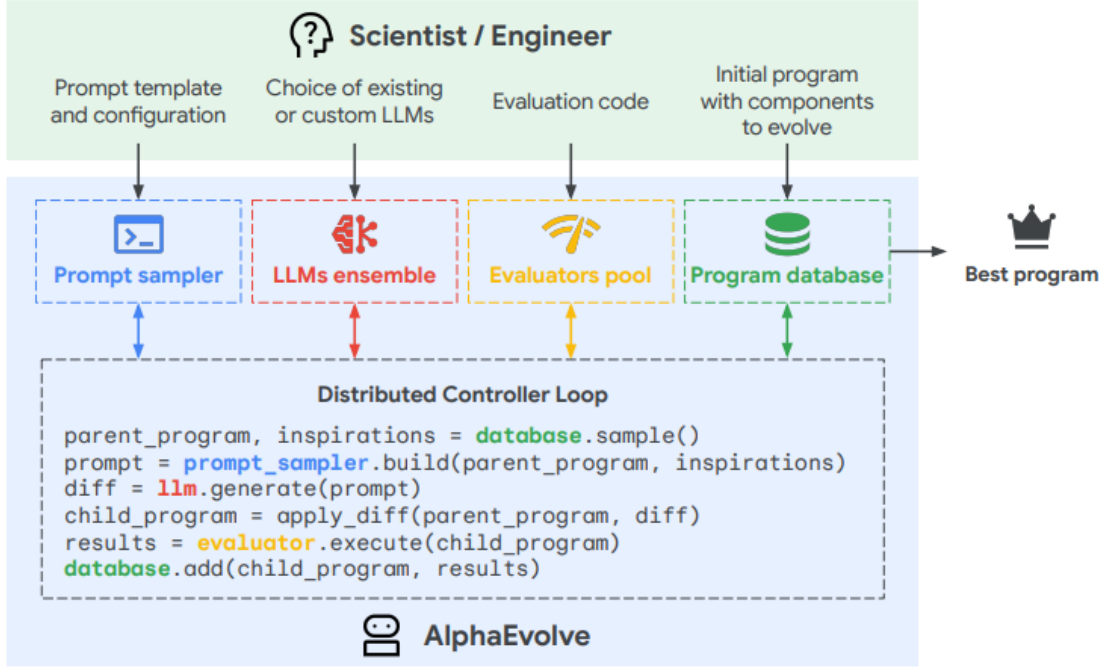


FIGURE 4.1: The AlphaEvolve program-evolution process, adapted from Novikov et al. [19].

4.3.4 Prompt sampling and creative generation

AlphaEvolve prompts contain more than the current program. Depending on the configuration, the context can contain high-scoring programs, structurally diverse programs, previous attempts, evaluator outputs, artifacts, explicit human-written instructions, and relevant examples or literature. The complete experiment-specific prompt composition is shown in Appendix A.1. This context allows an LLM to compare alternatives rather than repeatedly proposing a mutation without knowing what has already been tried.

The LLM is asked to make a concrete source-level change. For a diff-based mutation, the expected response is a sequence of blocks of the form

```

<<<< SEARCH
original code
=====
replacement code
>>>> REPLACE

```

The search part identifies the source segment to be replaced, and the replace part contains the candidate implementation. The complete mutation template is provided in Appendix A.1.2. A complete rewrite can be used for short programs or when a targeted change is not appropriate. In both cases, the model is instructed to preserve the public interface and to optimize the metrics defined by the evaluator.

AlphaEvolve uses an ensemble rather than relying on only one generation model. In the original system, a faster model increases the number of explored ideas, while a more capable model can supply less frequent but more substantial changes [19]. The ensemble is not an ensemble in the sense of averaging source code. A single model response is selected for each mutation, after which the resulting program is evaluated independently.

4.3.5 Evaluation, evolutionary storage, and distributed execution

AlphaEvolve supports several mechanisms for making the evaluation stage both useful and affordable. An evaluation cascade can apply inexpensive tests first and run expensive tests only for candidates that pass earlier thresholds. Parallel test cases can be distributed across workers. Multiple metrics can be returned simultaneously, both because they may represent genuine objectives and because programs that perform well under different metrics can provide more varied context for future mutations. Optional LLM feedback can be used for properties that are difficult to express in a conventional evaluator, such as readability or simplicity.

The system is implemented as an asynchronous pipeline. LLM generation and evaluation are started whenever their required inputs are available, and the controller waits only at dependencies between stages. This maximizes the number of candidates evaluated within a fixed computation budget instead of minimizing the latency of an individual candidate.

4.3.6 Scope and limitations of AlphaEvolve

The unique feature of AlphaEvolve is the combination of open-ended LLM mutations with machine-gradeable selection. It is not a general replacement for a software engineer, and it cannot be applied directly when candidate quality cannot be measured by an evaluator. Its results depend on the quality, coverage, and resistance to exploitation of the evaluation function. A weak metric can reward a program that optimizes the measurement rather than the intended task. The search can also require substantial computation because many candidates must be generated and evaluated, and there is no general guarantee that the best program will be found within a finite budget.

4.4 OpenEvolve

4.4.1 Open-source implementation

OpenEvolve is an open-source implementation of the AlphaEvolve design [26] and is used as the execution framework for all surface-realization experiments. The implementation exposes the same basic interface as AlphaEvolve: an initial program, an evaluator, and a configuration determine an iterative process in which programs are proposed, scored, and registered for future use.

The main controller loads the initial source, initializes the LLM ensemble, prompt sampler, evaluator, and program database. The initial program is evaluated before the evolutionary workers are started. On a fresh run it is then inserted into the database as generation zero. This makes the baseline available to the selection mechanism and ensures that evolution starts from a measured candidate rather than an unscored source file.

4.4.2 Execution architecture used in the thesis

The local implementation uses parallel workers for process-based parallelism. Before a worker is submitted, the controller selects a parent program and inspiration programs, which are selected the same way as the parent program, and are used as a source of inspiration for the LLM. Then the controller builds the prompt, calls the LLM, parses the response, and evaluates the resulting source.

Consequently, the logical evolution loop is:

1. select a parent and inspirations;
2. assemble a prompt from the selected context;

3. generate and parse an LLM mutation;
4. execute the candidate evaluator;
5. register the candidate, metrics, and feedback;
6. repeat until the iteration budget, target score, or stopping rule is reached.

The implementation is asynchronous in practice. Several workers can use snapshots of the program database made close together, and their results can be incorporated in a different order from the numerical iteration identifiers. This increases throughput but means that a candidate does not necessarily use the immediately preceding candidate as its parent.

4.4.3 Extensions made for the experiments

Several changes were made to the OpenEvolve’s algorithm. They address both the search distribution and the failure modes that are common when a language model is required to emit an exact source-code patch.

Fitness-weighted and island-specific selection. The database was extended with roulette-wheel selection. Parent and inspiration programs can be sampled with probabilities derived from their fitness instead of being selected uniformly. A small positive lower bound is used in the fitness weights so that low-scoring programs are not removed from consideration immediately. LLM configurations can also be assigned to islands, allowing a population to be associated with a particular model or model mixture. An optional iteration-based model schedule was added for periodically forcing a chosen model.

Configurable inspirations. The number of inspiration programs that are selected and added to the prompt can be controlled.

More robust SEARCH/REPLACE application. The original exact line matching was extended with trailing-whitespace cleanup and common-indentation handling. A model can therefore return a source block with a different outer indentation while the replacement is still inserted at the correct nesting level. If the response has no parsable diff, a corrective prompt asks the LLM to return the required format. If a syntactically valid diff does not change the source because its SEARCH block does not match, a second corrective prompt includes the parent code and asks the LLM to repair the SEARCH section. The exact repair prompts are given in Appendix A.1.5.

Retention of failed candidates. Malformed or unchanged proposals are written to a separate directory instead of being silently discarded. Duplicate source code is detected before insertion. These changes prevent repeated evaluation of identical candidates and preserve evidence needed to diagnose the behavior of the LLM and the mutation parser.

Explicit zero scores for timeouts. The evaluator integration was changed so that timeout and failed-evaluation paths give a score of 0.

4.5 Program database and initial program

4.5.1 Program records and persistence

The program database is implemented in a way to store those principal fields:

- a unique identifier and the program source;
- the identifier of the parent program, generation number, timestamp, and iteration at which the program was found;
- a dictionary of evaluation metrics, including the fitness metric;
- derived values such as complexity and diversity and metadata such as island assignment and a human-readable change summary;
- optional prompts and LLM responses associated with its creation;
- optional artifacts and embeddings used for feedback and novelty checks.

The active database is maintained in memory for fast selection. For each program, feature coordinates are calculated before insertion.

At most one program is retained as the occupant of a feature cell. If a new program maps to an empty cell, the cell is occupied. If it maps to an occupied cell, the program with the better fitness is retained as the cell occupant. The database separately stores a bounded set of elite programs. The absolute best program is tracked independently so that it is protected from ordinary population cleanup. When the configured population limit is exceeded, the lowest-fitness unprotected programs are removed from the active in-memory population, together with their references in islands and cells. The main limits are configurable.

4.5.2 Initial program

The initial program, reproduced in Appendix C, defines a small `Triple` data class with `subject`, `predicate`, and `object` fields. Its evolution block contains the following baseline behavior: `predict` receives a list of triples and returns a generic sentence for the first triple only:

The {predicate} of {subject} is {object}.

This implementation is intentionally minimal. It is syntactically valid and satisfies the interface, but it ignores additional triples, treats every predicate identically, and does not model relations between entities.

The purpose of evolution is not merely to make this baseline longer. The desired search direction is from a generic fallback sentence towards a domain-aware surface realizer. Useful changes include mapping predicates to natural-language templates, grouping triples by subject, identifying a main entity, ordering attributes, joining related entities, selecting relative clauses, and maintaining grammatical agreement. These changes are evaluated by the generated text rather than being prescribed as a fixed hand-written algorithm, which allows multiple realization strategies to be explored.

4.6 Program selection and prompt engineering

4.6.1 Parent and inspiration selection

Selection is performed before a worker process is launched. For roulette selection, the parent is selected from the island using a fitness-weighted softmax distribution. The use of a distribution rather than always selecting the best program retains a non-zero probability of returning to less explored solutions.

Inspirations are selected separately from the parent. They are alternative programs that can provide a design idea without being the source that is directly modified. With roulette, their probabilities are also based on fitness, and the parent itself is excluded from the candidate inspiration set. The selected parent and inspirations are passed to the worker, which reconstructs their source code from the database.

4.6.2 Prompt composition

The system message used for the surface-realization experiment is reproduced in Appendix A.1.1. It contains four kinds of fixed information:

- the role of the model as an expert data engineer;
- the description of the data to be processed and the required `predict` function signature;
- an example in which several triples are combined into one complex sentence;
- the list of predicates that may occur, with one example triple for each predicate.

The user message is assembled for each selected parent. Its complete template and runtime components are shown in Appendix A.1.2. Its main parts are:

- the current metrics and fitness score;
- optional evaluator artifacts, such as low-scoring triples and the generated text that caused them (see Appendix A.1.4);
- the complete current source code;
- the mutation task, including the requirement to improve the score, preserve the interface, and use the exact `SEARCH/REPLACE` format.

Consequently, the code and score of the selected parent are shown directly in the current-program section. An inspiration is rendered separately with its own score and source code. A high-scoring program can therefore provide a strong implementation pattern, while a structurally different inspiration can suggest a different way of grouping or ordering triples.

4.6.3 `SEARCH/REPLACE` instructions and their extensions

The mutation prompt asks for a targeted edit rather than a complete rewritten file. The `SEARCH` section must identify source text present in the current program, and the `REPLACE` section must contain the proposed replacement. The controller extracts all matching blocks with a regular expression, applies them line by line, and checks whether the resulting source differs from the parent. This format makes changes auditable and reduces the risk that the LLM rewrites fixed parts of the program unintentionally. The full mutation prompt is given in Appendix A.1.2.

The format is also a significant source of failure. A model can describe a patch in prose, omit one of the delimiters, copy an outdated version of the source, or use an indentation that prevents an exact match. These problems are addressed in two ways. First, trailing whitespace is normalized and common indentation is removed while matching, after which the replacement is re-indented at the original location. Second, malformed responses and unapplied responses are sent to dedicated repair prompts. These safeguards preserve the replacement text where possible and focus the subsequent LLM call on producing a matchable SEARCH section; the repair messages are listed in Appendix A.1.5.

4.7 Large Language Model

4.7.1 Parsing the LLM response

OpenEvolve uses an OpenAI-compatible client interface, so the same orchestration code can communicate with hosted models or local servers. The response is processed in the worker in the following order:

1. The response is searched for one or more SEARCH/REPLACE blocks.
2. Each block is normalized into source lines and applied to the parent source.
3. If at least one block is found but no source line is changed, the response is treated as an unapplied mutation rather than as a successful no-op.
4. A new identifier is assigned and the candidate is passed to the evaluator.

When no valid block is found, the response is not immediately treated as a failed iteration. Up to four attempts are made. The first repair instruction explains the required delimiter format. If a block was parsed but could not be applied, the repair instruction additionally contains the original parent code and asks the model to correct the SEARCH section while preserving the replacement. The intermediate invalid responses are joined and retained as diagnostic feedback.

If all repair attempts fail, a child record carrying the unchanged parent code is returned with an error and a `no_valid_diffs` or `no_changes` artifact. The controller writes this record to the list of broken programs and does not add it to the active evolutionary population.

OpenEvolve also contains a full-rewrite mode. When diff-based evolution is disabled, a fenced code block or the plain response is parsed as the complete candidate source. This mode was retained for compatibility, but the surface realization experiments use targeted diffs because the evolved function is embedded in a fixed program skeleton.

4.7.2 Invalid programs and execution safety

A generated program can fail before it produces any text. Dynamic loading can fail because of a syntax error or missing import; the required `predict` function can be absent or have the wrong signature; and a call can return a non-string value, raise an exception, or run indefinitely. These cases are converted into zero-valued fitness and artifacts describing the failure rather than terminating the entire evolution run. The artifacts can then be included in a later prompt, where they provide a concrete example of what should be avoided.

4.8 Evaluation

4.8.1 Evaluation and feedback

The OpenEvolve **Evaluator** is a generic wrapper around the task-specific evaluation file. For every candidate, it invokes the function named `evaluate`. The result contains metrics and artifacts. The wrapper applies the configured timeout, handles retries, converts both result formats to a common representation, and collects pending artifacts for the program record.

The initial program is passed through the same evaluator before evolution starts. This establishes the baseline against which later candidates are compared. A worker follows the same path after it has applied an LLM mutation.

The task-specific evaluator is therefore the boundary between source-code generation and evolutionary selection: OpenEvolve does not infer whether a sentence is good from the source code itself. It returns a metric dictionary and optional *artifacts*, which are stored diagnostic messages rather than values used directly for ranking. The metric named `combined_score` is the scalar fitness used by the database. Two mutually exclusive fitness modes are used:

- BLEU score evaluation; evaluates reference text with the generated text;
- LLM as a judge evaluation; evaluates the generated text with an LLM judge that compares the reference text with the generated text.

The code can calculate BLEU in judge-based runs as a diagnostic, but only the selected mode supplies fitness.

4.8.2 Reference-based evaluation

The surface-realization evaluator loads a supplied reference data corpus. Each selected entry contains a triple set and one or more reference lexicalizations.

Before the examples are processed, the evaluator loads the candidate module dynamically and checks that `predict` exists and accepts exactly one argument. For each reference data entry, the list of triples is passed to `predict`. A result is accepted only when it is a string. An empty string receives a per-example zero lexical score. Exceptions, invalid return types, and timeouts terminate the affected evaluation path and produce zero-valued metrics with an explanation in the artifacts.

For every non-empty generated text, the evaluator computes a fitness score (which is a BLEU score or LLM as a judge evaluation) against all reference lexicalizations associated with the entry. The average over the selected domain entries is returned as an aggregate reference-based measure.

4.8.3 Artifacts and continuation

An artifact is a per-candidate diagnostic message retained alongside its metrics. In BLEU-driven runs, a low-scoring artifact contains the input triples, generated text, reference lexicalizations, and the BLEU result. In judge-driven runs, it contains the triples, generated text, reference text, judge review, and rating. The worst scored artifacts are retained and three of them are randomly selected for inclusion in the next prompt.

After evaluation, the controller registers the child source, parent identifier, metrics, and artifacts, updates the population, archive, feature-cell representative, and best-program record, and submits another iteration. Optional trace files additionally record this provenance for later inspection. The repeated sequence of mutation, execution, scoring, and registration is what turns LLM code generation into the surface-realization search used in this thesis.

Chapter 5

Experiments

5.1 Meaning representation

5.1.1 Data Collection

The experimental evaluation for meaning representation requires structured data from diverse domains. To obtain such data, *quintd* [12] is used, a framework for collecting *ad hoc* data-to-text datasets. Quintd selects examples using a random seed and can also obtain up-to-date information from external data sources.

The source collection used in the experiments is denoted by S_0 . It contains the five domains available in the corresponding quintd collection: GSMArena, OpenWeather, Our World in Data (OWID), Wikidata, and ice hockey. The automatic pipeline comparison uses all five domains. The human evaluation, fine-tuning, and whole-system experiments use the four-domain subset consisting of GSMArena, OpenWeather, OWID, and Wikidata. This distinction is important because ice hockey is included in the automatic pipeline results but is absent from the later S_1 , S_2 , S_3 , and S_4 datasets.

The four experimental datasets are:

GSMArena Mobile phone specifications scraped from the GSMArena website. Each instance describes a single phone model with nested categories such as Network, Launch, Body, Display, Platform, Memory, Camera, and Battery, containing attribute–value pairs.

OpenWeather Five-day weather forecasts obtained from the OpenWeather API. Each instance contains a time series of 40 three-hour intervals, with each interval recording temperature, humidity, wind speed, cloud coverage, and weather conditions.

OWID Country-level time-series metrics from the Our World in Data project. Each instance represents a single metric, such as life expectancy, vaccination rates, or reproduction rate, for a specific country and contains yearly data points spanning several decades.

Wikidata Knowledge-graph entities and their properties from Wikidata. Each instance describes an entity, such as a country, film, or organization, with a list of property–value pairs such as capital, cast member, or language used.

The fifth domain, *ice hockey*, is therefore included only in the automatic pipeline comparison. Its absence from the human evaluation, fine-tuning, and whole-system experiments should not be interpreted as an omission from the source collection.

All experimental datasets are provided as JSON files containing arrays of structured records. A *raw structured instance* is defined as a single JSON object representing one self-contained data record from one of the four experimental domains. Each instance contains nested fields with structured data—time series, categorical values, numerical measurements, or hierarchical attribute lists—that must be converted into semantic triples. Let $D = \{d_1, \dots, d_N\}$ denote the set of N such instances extracted from the input JSON for a given domain.

Large language model. All LLM invocations across the three conversion methods use *Gemma 4 31B IT* [27, 28], a 31-billion-parameter instruction-tuned language model released by Google. The model is served via *vLLM* [15], a high-throughput inference engine that uses *PagedAttention* to manage key–value cache memory and enable batched inference with dynamic sequence lengths.

5.1.2 Experimental setup for pipelines

The automatic pipeline comparison uses the S_0 source collection of 100 instances from each of the five experimental domains. The reference-text generation stage is shared by the three pipelines, while the text-to-triples stage is run separately for Text-First Extraction with Normalization, Blueprint-Iterative Refinement, and Evolving Catalog.

The generated outputs are evaluated using a human evaluation and an LLM as a judge. The judge is the same Gemma 4 31B IT model, but it is used in a separate evaluation role rather than as part of the extraction pipeline. Two transitions are assessed independently:

1. the conversion from the raw structured instance to the generated natural-language reference text; and
2. the conversion from the reference text to the generated semantic triples.

The two transitions use different pairs of criteria, with the complete prompts provided in Appendix A.2.1. The data-to-text transition is assessed for *summary* and *faithfulness*. Summary measures whether the generated text is concise, coherent, non-redundant, and preserves the core message of the structured source. Faithfulness measures whether every textual claim is supported by that source, penalizing hallucinations, contradictions, and material distortions. The text-to-triples transition is assessed for *additions* and *omissions*. Additions measures whether the triples introduce facts not supported by the reference text, whereas omissions measures whether they leave out information whose absence changes meaning, hides important context, or creates a misleading imbalance. Every criterion is scored on an ordinal scale from 1 to 5, where 5 represents the best result.

The shared data-to-text evaluation requires $5 \times 100 \times 2 = 1,000$ judge calls. The three text-to-triples methods require $5 \times 100 \times 3 \times 2 = 3,000$ further calls, for 4,000 calls in total. For each domain and criterion, scores are averaged over the 100 instances. The resulting averages are used to compare the conversion methods.

The evaluation output also contains descriptive statistics. These statistics are computed without additional LLM calls and describe the size and structure of the generated outputs: the number of primitive JSON elements, reference words, reference sentences, reference subsentences, extracted triples, and unique predicates. The overall text and triples scores are calculated as the mean of the two criteria belonging to the corresponding transition.

TABLE 5.1: Automatic data-to-text evaluation shared by the three pipelines.

Domain	Summary	Faithfulness	Overall	JSON elements
Ice hockey	5.000	4.950	4.970	111.67
GSMARena	5.000	5.000	5.000	111.07
OWID	4.990	4.930	4.960	478.06
OpenWeather	5.000	5.000	5.000	902.92
Wikidata	4.990	4.990	4.990	13.18
Average	4.996	4.974	4.984	323.38

TABLE 5.2: Automatic text-to-triples evaluation for Blueprint-Iterative Refinement.

Domain	Additions	Omissions	Overall	Ref. words	Ref. sent.	Ref. subsent.	Triples	Predicates
Ice hockey	3.580	3.380	3.480	34.07	1.35	4.26	4.79	12
GSMARena	4.960	3.840	4.400	70.74	3.34	9.05	12.96	25
OWID	4.940	4.810	4.880	57.18	2.27	6.22	15.87	8
OpenWeather	4.880	3.840	4.360	66.21	2.82	6.53	5.46	18
Wikidata	4.910	4.410	4.660	20.34	1.17	2.61	4.15	47
Average	4.654	4.056	4.356	49.71	2.19	5.73	8.65	22.0

TABLE 5.3: Automatic text-to-triples evaluation for Text-First Extraction with Normalization.

Domain	Additions	Omissions	Overall	Ref. words	Ref. sent.	Ref. subsent.	Triples	Predicates
Ice hockey	5.000	4.970	4.990	32.54	1.32	4.21	6.77	106
GSMARena	5.000	5.000	5.000	70.34	3.26	9.30	13.61	159
OWID	5.000	4.990	5.000	58.23	2.28	6.31	8.09	281
OpenWeather	5.000	5.000	5.000	66.54	2.83	6.65	10.78	349
Wikidata	4.970	5.000	4.990	20.00	1.15	2.55	4.22	111
Average	4.994	4.992	4.996	49.53	2.17	5.80	8.69	201.2

TABLE 5.4: Automatic text-to-triples evaluation for Evolving Catalog.

Domain	Additions	Omissions	Overall	Ref. words	Ref. sent.	Ref. subsent.	Triples	Predicates
Ice hockey	4.990	3.920	4.460	31.88	1.27	4.10	5.14	4
GSMARena	4.970	4.810	4.890	69.48	3.19	9.11	12.87	25
OWID	4.880	3.850	4.370	56.55	2.22	5.99	3.40	139
OpenWeather	4.830	4.320	4.580	65.62	2.77	6.77	8.84	10
Wikidata	4.810	4.780	4.790	20.19	1.14	2.62	3.69	45
Average	4.896	4.336	4.618	48.74	2.12	5.72	6.79	44.6

5.1.3 Human evaluation for pipelines

Human evaluation follows the same asymmetric rubric as automatic evaluation. The shared data-to-text transition is assessed for summary and faithfulness, while each text-to-triples pipeline is assessed for additions and omissions. For each of the four domains, ten instances are assessed for every relevant transition. Scores are assigned by the author of the thesis on the same 1–5 scale as automatic evaluation.

TABLE 5.5: Human evaluation of the shared data-to-text transition.

Domain	Summary	Faithfulness	Overall
GSMArena	4.10	3.60	3.85
OWID	4.70	4.80	4.75
OpenWeather	4.60	3.60	4.10
Wikidata	4.60	4.80	4.70
Average	4.50	4.20	4.35

TABLE 5.6: Human evaluation of Blueprint-Iterative Refinement.

Domain	Additions	Omissions	Overall
GSMArena	4.30	3.70	4.00
OWID	4.00	4.30	4.15
OpenWeather	5.00	3.00	4.00
Wikidata	4.80	4.30	4.55
Average	4.525	3.825	4.175

TABLE 5.7: Human evaluation of Text-First Extraction with Normalization.

Domain	Additions	Omissions	Overall
GSMArena	4.90	5.00	4.95
OWID	5.00	4.50	4.75
OpenWeather	4.90	5.00	4.95
Wikidata	5.00	5.00	5.00
Average	4.950	4.875	4.913

TABLE 5.8: Human evaluation of Evolving Catalog.

Domain	Additions	Omissions	Overall
GSMArena	4.90	4.90	4.90
OWID	5.00	2.20	3.60
OpenWeather	4.90	3.70	4.30
Wikidata	4.80	4.80	4.80
Average	4.900	3.900	4.400

5.1.4 Correlation between LLM and human evaluation

The agreement between the LLM judge and the human evaluator is computed from paired 1–5 scores.

Four complementary statistics are reported. Pearson’s product–moment correlation measures linear association between the raw values, but assumes equal distances between the ordinal score levels. Spearman’s ρ measures whether the LLM and human rank instances similarly and is the primary association measure for the ordinal 1–5 scale. Kendall’s τ -b measures pairwise rank concordance while correcting for ties, making it useful for the small samples and repeated ratings in this experiment. Quadratic weighted Cohen’s kappa measures chance-corrected ordinal agreement and penalizes a two-point disagreement more strongly than a one-point disagreement [4]. All four measures range from -1 to 1 ; positive values indicate agreement or positive association, zero indicates no association or agreement beyond chance, and negative values indicate inverse association.

The overall variants are calculated separately for the two evaluation tasks and never mix data-to-text with text-to-triples. A *pooled* text-to-triples result concatenates the additions and omissions score of every matched instance for each rater. With ten matched instances, this produces 20 paired criterion scores: additions and omissions for each instance. It preserves criterion-specific disagreement, for example when a judge agrees on additions but not on omissions.

The complementary *per-instance mean* result first averages additions and omissions for each rater within an instance and then correlates the resulting ten overall-quality pairs. It answers whether the LLM and human agree about which complete instances are better or worse overall, while deliberately discarding criterion-specific differences. For this mean variant, Pearson, Spearman, and Kendall are computed from the floating-point means; quadratic weighted kappa uses the means rounded to the nearest ordinal score.

The *LLM score mean* and *Human score mean* columns report the arithmetic mean of the aligned score vector used for that row. For an additions or omissions row, each mean is calculated from the ten scores for that single criterion. For a pooled row, it is calculated from 20 scores: additions and omissions for each of the ten instances. For a per-instance-mean row, it is calculated from ten instance-level averages. These columns describe the rating level of each evaluator and are not correlation coefficients.

Pearson, Spearman, and Kendall are undefined when either rater assigns the same score to every observation. The status column records such constant-vector cases. Means remain available for comparing score levels, but are not correlation coefficients. In particular, a kappa of zero in a constant-vector case indicates no measurable agreement beyond chance under the observed score distributions; it does not establish that every individual rating disagreed.

TABLE 5.9: Data-to-text correlation diagnostics ($N = 10$ instances per row).

Domain	Criterion	Pearson r	Spearman ρ	Kendall τ	QW- κ	LLM score mean	Human score mean	Status
GSMArena	Summary	–	–	–	0.000	5.00	4.10	LLM constant
GSMArena	Faithfulness	–	–	–	0.000	5.00	3.60	LLM constant
OWID	Summary	–	–	–	0.000	5.00	4.70	LLM constant
OWID	Faithfulness	–	–	–	0.000	5.00	4.80	LLM constant
OpenWeather	Summary	–	–	–	0.000	5.00	4.60	LLM constant
OpenWeather	Faithfulness	–	–	–	0.000	5.00	3.60	LLM constant
Wikidata	Summary	–	–	–	0.000	5.00	4.60	LLM constant
Wikidata	Faithfulness	–	–	–	0.000	5.00	4.80	LLM constant

TABLE 5.10: Text-to-triples correlation results by criterion and aggregation.

Pipeline	Domain	Criterion / aggregation	N	r	ρ	τ	QW- κ	LLM score mean	Human score mean	Status
Blueprint	GSMArena	Additions	10	-0.509	-0.509	-0.509	-0.212	4.90	4.30	ok
		Omissions	10	0.349	0.387	0.360	0.286	4.00	3.70	ok
		Pooled	20	0.402	0.371	0.344	0.316	4.45	4.00	ok
		Per-instance mean	10	0.000	0.000	0.000	0.000	4.45	4.00	ok
		Additions	10	—	—	—	0.000	5.00	4.00	both constant, different
		Omissions	10	0.512	0.520	0.494	0.314	4.80	4.30	ok
	OWID	Pooled	20	0.379	0.360	0.348	0.133	4.90	4.15	ok
		Per-instance mean	10	0.512	0.520	0.494	0.000	4.90	4.15	ok
		Additions	10	—	—	—	0.000	4.70	5.00	Human constant
		Omissions	10	0.588	0.529	0.494	0.462	3.50	3.00	ok
		Pooled	20	0.726	0.731	0.664	0.714	4.10	4.00	ok
		Per-instance mean	10	0.633	0.536	0.501	0.000	4.10	4.00	ok
	OpenWeather	Additions	10	0.667	0.667	0.667	0.615	4.90	4.80	ok
		Omissions	10	0.927	0.964	0.934	0.906	4.40	4.30	ok
		Pooled	20	0.895	0.906	0.886	0.873	4.65	4.55	ok
		Per-instance mean	10	0.907	0.868	0.841	0.800	4.65	4.55	ok
	Wikidata	Additions	10	—	—	—	0.000	5.00	4.90	LLM constant
		Omissions	10	—	—	—	0.000	5.00	5.00	both constant, same
		Pooled	20	—	—	—	0.000	5.00	4.95	LLM constant
		Per-instance mean	10	—	—	—	0.000	5.00	4.95	LLM constant
	OWID	Additions	10	—	—	—	0.000	5.00	5.00	both constant, same
		Omissions	10	—	—	—	0.000	5.00	4.50	LLM constant
		Pooled	20	—	—	—	0.000	5.00	4.75	LLM constant
		Per-instance mean	10	—	—	—	0.000	5.00	4.75	LLM constant
	OpenWeather	Additions	10	—	—	—	0.000	5.00	4.90	LLM constant
		Omissions	10	—	—	—	0.000	5.00	5.00	both constant, same
		Pooled	20	—	—	—	0.000	5.00	4.95	LLM constant
		Per-instance mean	10	—	—	—	0.000	5.00	4.95	LLM constant
	Wikidata	Additions	10	—	—	—	0.000	5.00	5.00	both constant, same
		Omissions	10	—	—	—	0.000	5.00	5.00	both constant, same
		Pooled	20	—	—	—	0.000	5.00	5.00	both constant, same
		Per-instance mean	10	—	—	—	0.000	5.00	5.00	both constant, same
Catalog	GSMArena	Additions	10	—	—	—	0.000	5.00	4.90	LLM constant
		Omissions	10	0.667	0.667	0.667	0.615	4.80	4.90	ok
		Pooled	20	0.444	0.444	0.444	0.444	4.90	4.90	ok
		Per-instance mean	10	0.375	0.375	0.375	0.375	4.90	4.90	ok
	OWID	Additions	10	—	—	—	0.000	4.60	5.00	Human constant
		Omissions	10	0.867	0.917	0.875	0.404	3.60	2.20	ok
		Pooled	20	0.604	0.630	0.578	0.559	4.10	3.60	ok
		Per-instance mean	10	0.627	0.795	0.707	0.043	4.10	3.60	ok
	OpenWeather	Additions	10	—	—	—	0.000	5.00	4.90	LLM constant
		Omissions	10	0.753	0.646	0.631	0.478	4.40	3.70	ok
		Pooled	20	0.782	0.647	0.632	0.630	4.70	4.30	ok
		Per-instance mean	10	0.591	0.410	0.387	0.000	4.70	4.30	ok
	Wikidata	Additions	10	—	—	—	0.000	5.00	4.80	LLM constant
		Omissions	10	0.667	0.667	0.667	0.615	4.80	4.80	ok
		Pooled	20	0.459	0.459	0.459	0.444	4.90	4.80	ok
		Per-instance mean	10	0.804	0.646	0.626	0.412	4.90	4.80	ok

5.1.5 Experimental setup for fine-tuning

The fine-tuning experiments are conducted on the same four domains as the pipeline comparison: GSMarena, OpenWeather, OWID, and Wikidata. The training-and-validation collection is downloaded using seed S_1 , while the held-out test collection is downloaded independently using seed S_2 .

The raw instances from the S_1 and S_2 collection are converted into reference text and semantic triples using the Text-First Extraction with Normalization pipeline. The aligned text-triple outputs are then used as the targets for a domain-specific model. For each domain, the resulting examples are split into 80% training data and 20% validation data. The S_2 collection remains separate and is used only for the final evaluation of the trained models. This split prevents the test instances from influencing either model fitting or training-configuration selection.

The fine-tuned model receives one structured instance and is trained to return both a natural-language reference and its semantic triples in one JSON output. The base model is Gemma 4 31B IT, and the adaptation is performed using QLoRA. The base weights remain frozen, while low-rank adapters are trained in the language-model attention and feed-forward projections. After training, the adapters are merged into the base model before evaluation.

5.1.6 Training Configuration

The fine-tuning comparison contains five configurations. The standard baseline uses three epochs. The *baseline-epoch1* variant keeps the baseline learning rate, adapter capacity, dropout, and weight decay, but trains for one epoch. The remaining variants change the learning rate, adapter capacity, or regularization in order to test their influence on the domain-specialized models.

All training is performed on NVIDIA A100-SXM4 GPUs with 40 GB of VRAM, using the PLGrid Athena cluster. The common effective batch size is 16, obtained from a per-device batch size of 1 and 16 gradient-accumulation steps. The optimizer is 8-bit paged AdamW, the learning-rate schedule is cosine, and the compute precision is bfloat16. The base model is loaded using 4-bit NF4 quantization with double quantization.

TABLE 5.11: QLoRA configurations used for fine-tuning.

Configuration	Epochs	LR	Warmup	r	α	Dropout	Weight decay
baseline	3	10^{-4}	0.03	16	32	0.05	0.00
baseline-epoch1	1	10^{-4}	0.03	16	32	0.05	0.00
low_lr	5	3×10^{-5}	0.05	16	32	0.05	0.01
higher_capacity	3	5×10^{-5}	0.05	32	64	0.05	0.01
regularized_capacity	5	3×10^{-5}	0.10	32	64	0.10	0.01

The LoRA configuration targets all attention and MLP projection matrices in the language-model backbone (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`). A positively scoped module pattern excludes the vision and audio towers of the multimodal checkpoint. This selects 420 target modules and approximately 25 million trainable parameters, which is less than 0.1% of the base model.

The maximum sequence length is adjusted per domain to accommodate the varying sizes of the raw structured instances while staying within the GPU memory budget. Wikidata uses 8192 tokens, GSMarena and OWID use 4096 tokens, and OpenWeather uses 6144 tokens. Instances exceeding the selected maximum are truncated.

After training, the LoRA adapter weights are combined with the frozen base model weights to produce a standalone model. This process, computes $W_{\text{final}} = W_0 + \Delta W$ for each adapted layer.

The merged model can be served by the same vLLM infrastructure as the base model and produces both a natural-language verbalization and the corresponding semantic triples in a single decode pass.

5.1.7 Evaluation of fine-tuned models

The fine-tuned models are evaluated on the held-out S_2 test sets for each domain. The evaluation compares the generated text with the reference text using BLEU and METEOR. The generated and reference triples are compared as sets of subject–predicate–object tuples. Micro-averaged triple precision, recall, and F1 are computed from the resulting true-positive, false-positive, and false-negative counts. Predicate adherence measures the proportion of generated predicates that belong to the predicate catalog used to construct the targets.

The results are reported for the base model, which is the Gemma 4 before any fine-tuning and separately for the five fine-tuning configurations.

TABLE 5.12: Base-model results on the held-out S_2 sets.

Domain	Text BLEU	Text METEOR	Triple precision	Triple recall	Triple F1	Predicate adherence
GSMARena	0.5621	0.7455	0.0515	0.0413	0.0458	0.3863
OpenWeather	0.2574	0.4931	0.0013	0.0010	0.0011	0.3771
OWID	0.3919	0.6170	0.0000	0.0000	0.0000	0.4181
Wikidata	0.7474	0.9032	0.0305	0.0409	0.0349	0.3280
Average	0.4897	0.6897	0.0208	0.0208	0.0205	0.3774

TABLE 5.13: Fine-tuned model results for the baseline configuration.

Domain	Text BLEU	Text METEOR	Triple precision	Triple recall	Triple F1	Predicate adherence
GSMARena	0.6245	0.8297	0.4234	0.4411	0.4321	0.7087
OpenWeather	0.2121	0.4463	0.0010	0.0005	0.0007	0.2002
OWID	0.3096	0.5757	0.0245	0.0227	0.0236	0.7148
Wikidata	0.7642	0.9087	0.5137	0.4495	0.4795	0.6635
Average	0.4776	0.6901	0.2407	0.2285	0.2340	0.5718

TABLE 5.14: Fine-tuned model results for the baseline-epoch1 configuration.

Domain	Text BLEU	Text METEOR	Triple precision	Triple recall	Triple F1	Predicate adherence
GSMARena	0.5937	0.7969	0.2936	0.2942	0.2939	0.8389
OpenWeather	0.3175	0.5430	0.0013	0.0010	0.0011	0.3199
OWID	0.3745	0.6355	0.0090	0.0079	0.0084	0.6147
Wikidata	0.7136	0.8894	0.4103	0.1154	0.1801	0.7479
Average	0.4998	0.7162	0.1786	0.1046	0.1209	0.6304

TABLE 5.15: Fine-tuned model results for the low learning-rate configuration.

Domain	Text BLEU	Text METEOR	Triple precision	Triple recall	Triple F1	Predicate adherence
GSMARena	0.6399	0.8257	0.3645	0.3615	0.3630	0.7522
OpenWeather	0.3068	0.5421	0.0016	0.0010	0.0012	0.3033
OWID	0.3877	0.6457	0.0088	0.0078	0.0083	0.6047
Wikidata	0.7752	0.9104	0.4363	0.4531	0.4446	0.7280
Average	0.5274	0.7310	0.2028	0.2059	0.2043	0.5971

TABLE 5.16: Fine-tuned model results for the higher-capacity configuration.

Domain	Text BLEU	Text METEOR	Triple precision	Triple recall	Triple F1	Predicate adherence
GSMARena	0.6332	0.8316	0.4202	0.4310	0.4256	0.7306
OpenWeather	0.2448	0.4786	0.0027	0.0015	0.0019	0.1636
OWID	0.3654	0.6276	0.0256	0.0240	0.0248	0.6911
Wikidata	0.7513	0.9019	0.4757	0.4700	0.4728	0.6971
Average	0.4987	0.7099	0.2311	0.2316	0.2313	0.5706

TABLE 5.17: Fine-tuned model results for the regularized-capacity configuration.

Domain	Text BLEU	Text METEOR	Triple precision	Triple recall	Triple F1	Predicate adherence
GSMArena	0.6443	0.8297	0.3961	0.3953	0.3957	0.7373
OpenWeather	0.2957	0.5449	0.0025	0.0015	0.0018	0.2789
OWID	0.3911	0.6407	0.0191	0.0169	0.0179	0.6858
Wikidata	0.7555	0.8963	0.4912	0.4700	0.4803	0.6884
Average	0.5217	0.7279	0.2272	0.2209	0.2239	0.5976

5.2 Surface realization

5.2.1 WebNLG dataset and evaluation protocol

Surface realization is evaluated on WebNLG, a benchmark in which RDF-style semantic triples are paired with one or more natural-language lexicalizations. The benchmark reader used in this work loads the English WebNLG release and preserves both the triples and their reference texts. During evolution, the training portion of WebNLG is used to provide fitness feedback. The final programs are evaluated independently on the held-out test portion, preventing the test references from being used during program search [9].

The evaluation is performed separately by WebNLG domain. The program receives only the triples for each test entry. It must return one natural-language string. The baseline systems are a rule-based generator by Lango et al. [16], a fine-tuned BART model, and a prompted language model. The prompt can be found in Appendix A.2.

The final evaluation reports reference-based, judge-based, and descriptive metrics. BLEU, BLEURT, and METEOR compare a generated text with the WebNLG references. The three judge columns contain scores from Gemma 4 31B IT, Qwen 3.6, and Themis.

SENLEN is a thesis-specific structural diagnostic introduced for this evaluation. For a generated text realization $\hat{y}$ and its set of references R , it is defined as

$$\text{SENLEN}(\hat{y}, R) = \left| \text{sen}(\hat{y}) - \frac{1}{|R|} \sum_{r \in R} \text{sen}(r) \right|,$$

where $\text{sen}(x)$ is the number of sentences, which are counted using a period as the delimiter. This deliberately uses an implementation-level delimiter count rather than a linguistic sentence tokenizer. Lower values are preferable, and zero indicates that the generated realization has exactly the average reference sentence count.

The grammaticality, additions, and omissions judgments are performed with Gemma 4 31B IT. The prompts used for these judgments are provided in Appendix A.2. The final four columns are descriptive statistics: average sentence count, word count, character count, and number of input triples.

Higher values are preferable for BLEU, BLEURT, METEOR, judge scores, and grammaticality. Lower values are preferable for additions, and omissions; zero means that no addition or omission was detected. Lower SENLEN values are preferable; zero indicates that the period-delimited segment count exactly matches the average reference count.

5.2.2 Evolution variants

The initial all-domain run uses 1,000 iterations, Gemma 4 31B IT as the main program-generation model, BLEU fitness, and one inspiration program in each prompt. The later runs use 500 or 100 iterations and are restricted to three selected domains.

The initial run was used to identify domains with different levels of difficulty. WrittenWork was selected as the best-working domain, Building as an intermediate domain, and Food as the most difficult domain. Restricting subsequent variants to these three domains reduced the computational cost of the experiments while retaining distinct task difficulties.

The following variants were then evaluated:

BLEU without inspirations BLEU fitness and no inspiration programs. Uses Gemma 4 31B IT in non-reasoning mode for program generation.

BLEU with inspirations The preceding setup but with one inspiration program.

LLM judge without inspirations Same as the first setup but uses LLM-as-a-judge fitness.

LLM judge with inspirations The preceding setup but with one inspiration program.

Mixed-model LLM judge LLM-as-a-judge fitness and inspirations, with Gemma 4 31B used for 80% of calls and a DeepSeek-V3.1 685B model for 20% of calls.

Mixed-model LLM judge with four threads The preceding mixed-model setup with four parallel evaluator threads.

Mixed Gemma, four threads A 100-iteration mixed Gemma 4 31B run with 80% non-reasoning and 20% high-reasoning program-generation calls, and four parallel evaluator threads.

Mixed Gemma, one thread The preceding mixed-generation setup with one evaluator thread, isolating the effect of evaluator parallelism.

High-reasoning Gemma A 500-iteration Gemma 4 31B run in which every program-generation call uses high-reasoning generation, with four parallel evaluator threads.

Gemma E4B A 500-iteration single-thread run using Gemma 4 E4B for both program generation and LLM-as-a-judge fitness.

Llama 3.3 70B A 500-iteration single-thread run using Llama 3.3 70B for both program generation and LLM-as-a-judge fitness.

Qwen3 32B A 500-iteration single-thread run using Qwen3 32B for both program generation and LLM-as-a-judge fitness.

Standard Gemma with no references A 500-iteration single-thread Gemma 4 31B run with standard, non-reasoning program-generation calls, using an LLM-as-a-judge but this time without the text references.

The final eight variants use no inspiration programs and LLM-as-a-judge fitness. In the Gemma variants, Gemma 4 31B IT is the evaluator model; the model-comparison variants use the respective program-generation model as the evaluator.

Each run produces an executable surface-realization program. The program is then evaluated on the held-out WebNLG test set rather than being judged only by its online evolutionary fitness.

5.2.3 Evolution results

The following tables use the metric order shown in the evaluation protocol. Average rows are calculated over the available rows for the corresponding experiment.

5.2.4 Baseline systems

The baselines are a rule-based generator, a fine-tuned BART model, and an LLM prompt. The average is calculated over the available domain results for each baseline.

TABLE 5.33: Baseline scores across WebNLG domains.

method	domain	BLEU	BLEURT	METEOR	SENLEN	Gemma	Qwen	Themis	gram.	add.	om.	sent.	words	chars	triples
Rule-based generator	all domains	0.1702	-0.3874	0.5205	1.0175	0.4108	0.4811	0.7552	0.1783	0.0889	0.0742	2.5414	21.7471	146.6654	3.1946
Fine-tuned BART	all domains	0.5077	0.2599	0.7163	0.5157	0.6366	0.5933	0.7899	0.8366	0.3142	0.1863	2.1679	138.9440	23.7167	3.5205
LLM prompt	all domains	0.5019	0.3443	0.7246	0.5274	0.9954	0.9743	0.9847	0.9861	0.0045	0.0022	1.7307	132.0601	21.5757	3.5205

5.2.5 Three-domain comparison

The final comparison reports averages over WrittenWork, Building, and Food for the baselines and the three-domain OpenEvolve variants reported below. The all-domain initial variant is omitted because descriptive statistics were not reported for that run.

TABLE 5.34: Average scores on WrittenWork, Building, and Food.

method	BLEU	BLEURT	METEOR	SENLEN	Gemma	Qwen	Themis	gram.	add.	om.	sent.	words	chars	triples
Rule-based generator	0.2063	-0.4489	0.5498	1.9138	0.2316	0.3529	0.6380	0.0694	0.4291	0.1836	4.3887	30.8329	199.2032	3.3999
Fine-tuned BART	0.5628	0.2398	0.7126	0.7414	0.6707	0.5964	0.8123	0.7878	0.2782	0.1122	2.3025	147.8967	27.3637	3.8264
LLM prompt	0.4781	0.2310	0.6720	0.6706	0.9848	0.9660	0.9826	0.9778	0.0000	0.0000	2.2322	141.2164	25.2174	3.8264
BLEU, no inspirations	0.3521	0.1049	0.6314	0.6706	0.9286	0.7310	0.8484	0.3640	0.0725	0.0000	2.2322	27.1047	154.0871	3.8264

5.3 Whole system evaluation

The whole-system evaluation combines the meaning-representation and surface- realization stages into one experimental pipeline. Unlike the preceding experiments, which evaluate the two stages separately, this evaluation measures whether errors introduced while converting raw structured data into triples affect the final generated text.

5.3.1 Data construction

Two new collections are downloaded with `quintd` for each of the four domains: GSMArena, OpenWeather, OWID, and Wikidata. The first collection is denoted by S_3 and contains 1,000 raw structured instances per domain for training the evolutionary programs. The second collection is denoted by S_4 and contains another 1,000 raw structured instances per domain for final testing. The two collections are downloaded independently so that no raw instance from the test collection is available during evolution.

The raw instances in both collections are processed with the *regularized capacity* fine-tuned Gemma 4 model from the preceding fine-tuning experiments. For each instance, the model produces a natural-language reference and the associated normalized semantic triples in a single output. Thus, both S_3 and S_4 contain aligned pairs of the form

$$(T_i, r_i),$$

where T_i is the generated set of triples and r_i is the corresponding reference text. The model is selected before this evaluation and remains frozen while the whole-system experiments are performed. The generated references are therefore used as domain-specific evaluation targets, rather than as additional inputs to the surface-realization programs.

5.3.2 Evolution and evaluation protocol

For each domain, the selected best-working OpenEvolve variant is run using the S_3 collection. The evolved program receives only T_i and must generate a natural-language text. Its fitness is computed by comparing the generated text with r_i on the 1,000 training instances. A separate best program is retained for each domain. The S_4 triples and reference texts are not used during program generation, fitness calculation, or program selection.

After evolution, each domain-specific program is executed on all 1,000 instances in the corresponding S_4 test collection. The generated texts are evaluated using the same protocol as the WebNLG surface-realization experiments. BLEU, BLEURT, and METEOR compare the generated text with its reference. SENLEN measures the difference between the generated and reference period-delimited segment counts as defined in Section 5.2.1. Gemma 4, Qwen, and Themis provide judge scores, while grammaticality, additions, and omissions are evaluated using the corresponding judge prompts. Average sentence count, word count, character count, and number of input triples are reported as descriptive statistics.

In this experiment, the reference triples are produced by the selected fine-tuned model rather than annotated independently by humans. Consequently, the evaluation measures the quality of the complete data-to-text pipeline with respect to the distilled references. It does not separately estimate the semantic precision or recall of the fine-tuned model; those properties are reported in the fine-tuning experiments.

TABLE 5.35: Whole-system evaluation on the independent S_4 test collections.

[illegible]

Chapter 6

Discussion

This chapter interprets the separate evaluations of meaning representation and surface realization. The results support several practical conclusions, but they also show that the choice of method depends on the intended objective: semantic coverage, predicate control, reference similarity, judge-oriented faithfulness, or inspectability. Consequently, the discussion distinguishes observed results from explanations that remain hypotheses and from questions that require the pending end-to-end evaluation.

6.1 Meaning representation

6.1.1 Pipeline comparison

The pipeline results identify Text-First Extraction with Normalization as the most reliable of the three evaluated methods. Its automatic additions and omissions scores are 4.994 and 4.992, respectively, with an overall score of 4.996 in Table 5.3. The same ranking is retained in the human evaluation: its overall score is 4.913, compared with 4.400 for Evolving Catalog and 4.175 for Blueprint-Iterative Refinement in Tables 5.7, 5.8, and 5.6. This agreement between the two aggregate evaluations is useful evidence that the ranking is not solely an artifact of the automatic judge.

The result is consistent with the design of Text-First Extraction with Normalization. It allows each instance to be extracted independently, then normalizes the resulting predicates after extraction. This avoids constraining an instance to a vocabulary that was created from earlier cases. The method uses 201.2 unique predicates on average, compared with 44.6 for Evolving Catalog and 22.0 for Blueprint-Iterative Refinement (Tables 5.3, 5.4, and 5.2). A broader vocabulary is compatible with the semantic diversity of the input data and is plausibly one reason why fewer facts are omitted. However, the predicate count is a descriptive result rather than a controlled ablation; it cannot establish that vocabulary size alone causes the observed quality difference.

The errors of the other two pipelines illustrate the cost of early vocabulary control. Evolving Catalog is second overall, but its automatic omission scores drop to 3.920 for ice hockey and 3.850 for OWID. Its human omission score is particularly low for OWID at 2.20 and is also reduced for OpenWeather at 3.70 (Tables 5.4 and 5.8). The stable catalog provides a coherent and auditable vocabulary, but facts that do not fit its frozen or slowly expanding catalog can be lost. Blueprint-Iterative Refinement, provides the strongest up-front specification and therefore the clearest human-inspectable artifact. Its omission results nevertheless show the limits of a compact predetermined blueprint: automatic scores are 3.380 for ice hockey and 3.840 for OpenWeather, while the human omission mean is 3.825 (Tables 5.2 and 5.6).

The shared data-to-text stage is close to the ceiling of the automatic scale: the mean summary and faithfulness scores are 4.996 and 4.974 in Table 5.1. The human evaluation is more critical, with means of 4.50 and 4.20 in Table 5.5. In particular, GSMarena and OpenWeather both receive a human faithfulness mean of 3.60. This indicates that their generated summaries contain meaningful faithfulness issues under the evaluation rubric, but the mean should not be read as the rating of a single instance. It aggregates ten author-assigned ratings per domain. The gap between automatic and human means therefore motivates a cautious interpretation of the apparently near-perfect automatic scores.

6.1.2 Fine-tuning as pipeline distillation

Fine-tuning substantially improves the model’s ability to produce triples from raw data. The base Gemma model obtains a mean triple F1 of only 0.0205, whereas the three-epoch baseline reaches 0.2340 (Tables 5.12 and 5.13). One epoch already improves F1 to 0.1209, showing that the task-specific targets provide a strong adaptation signal even with limited training. At the same time, the results do not select a single configuration as best for every purpose. The three-epoch baseline has the highest reported average triple F1, the one-epoch model has the highest predicate adherence, and the low-learning-rate configuration has the highest average text BLEU and METEOR, at 0.5274 and 0.7310 (Tables 5.14 and 5.15). The regularized-capacity configuration is competitive for text realization, but is not the leader on either of these text metrics or on triple F1.

The domain results are more informative than the averages alone. Fine-tuning raises triple F1 markedly for GSMarena and Wikidata, while OpenWeather remains close to zero for every configuration. The best reported OpenWeather F1 is only 0.0019 in the higher-capacity configuration. OWID also remains substantially weaker than GSMarena and Wikidata. These differences are consistent with the greater structural variation and compression required by weather forecasts and statistical data, but the present experiments do not isolate which properties of those domains cause the difficulty. They show that per-domain model selection and additional domain-specific data work are needed rather than that one hyperparameter setting transfers uniformly.

The fine-tuned model should also be understood as a distillation mechanism, not as independent semantic supervision. Its targets are the reference texts and normalized triples produced by the upstream pipeline. It can make the inferred mapping from raw data to text and triples cheaper to execute in one decode pass, but it is bounded by the semantic quality and coverage of those targets. Better fine-tuning results therefore do not by themselves validate the underlying triples against the original raw data. Improving target construction and adding independently annotated examples remain necessary steps for establishing such validity.

6.2 Surface realization

6.2.1 Evolutionary fitness and domain effects

The initial all-domain evolution run shows a clear domain difference. WrittenWork has the highest BLEU score (0.5920), whereas Food has the lowest (0.0764), as shown in Table 5.18. WrittenWork may be easier because its triples are often amenable to recurring bibliographic descriptions, whereas food facts can require more varied lexical choices and combinations of attributes. This is an interpretation rather than a demonstrated causal result: the experiment does not separately measure lexical diversity, triple topology, or reference variability. Nevertheless, the contrast justifies the subsequent use of WrittenWork, Building, and Food as relatively easy, intermediate, and difficult domains.

The three-domain variants demonstrate a trade-off between the selection signal used during evolution and the metric used after evolution. The BLEU-fitness variant without inspirations has the best BLEURT value among the reported evolved programs, at 0.1049. Its paired variant with inspirations is slightly higher on BLEU, 0.3526 versus 0.3521, but lower on BLEURT, 0.0952 versus 0.1049 (Tables 5.19 and 5.20). Thus, removing inspirations cannot be claimed to improve BLEU consistently; it improves BLEURT in this direct comparison, while the BLEU difference is negligible and favours inspirations.

Variants optimized with LLM-as-a-judge fitness obtain stronger final judge scores and lower detected additions and omissions than the BLEU-fitness variants. For example, the no-inspiration LLM-judge variant has zero additions and omissions and receives a Gemma judge score of 1.0000 (Table 5.21). The mixed-model variants also report zero additions and omissions. This pattern is expected when evolutionary selection directly rewards judge-oriented properties, but it should not be interpreted as independent proof of semantic correctness. The LLM-judge-with-inspirations variant still has nonzero additions, and the reference-based BLEU, BLEURT, and METEOR values do not improve uniformly. The selection objective therefore shapes the type of quality found by evolution; it does not remove the need for external evaluation.

6.2.2 Comparison with baselines

The baseline comparison further emphasizes that there is no single dominant surface-realization approach. The rule-based generator is clearly weaker than the other systems across the reported measures. On the three selected domains, the fine-tuned BART baseline has the best reference-based scores: BLEU 0.5628, BLEURT 0.2398, and METEOR 0.7126. The zero-shot LLM prompt has lower reference-based scores, but it leads the three judge scores, grammaticality, and detected additions and omissions, the latter two both being zero (Table 5.34). It is the strongest reported system for judge-oriented faithfulness and fluency measures.

The reported evolved program does not yet exceed these stronger neural and prompted baselines on the aggregate reference metrics. Its value lies elsewhere: after evolution, the realization procedure is an inspectable Python artifact that can be executed on unseen triple sets without another LLM call. This property supports auditability, targeted modification, and potentially cheap CPU execution. It does not, however, demonstrate a quality or cost advantage in every deployment setting. A direct prompt is operationally simpler and is currently more effective on several judged measures; BART remains stronger on the principal reference metrics. The evolutionary approach is most compelling where a reusable and inspectable realization program is a requirement, rather than where the only objective is the highest benchmark score.

6.3 Evaluation validity and limitations

The judge-human comparison provides a necessary qualification to all judge-based conclusions. For data-to-text, the LLM judge assigns a constant score of 5 to every assessed instance, so Pearson, Spearman, and Kendall correlations cannot be calculated. The corresponding quadratic weighted kappa values are zero despite the lower human means (Table 5.9). The automatic judge therefore fails to distinguish the errors identified by the human evaluation in this near-ceiling regime.

Text-to-triples agreement is more informative but remains mixed. For Evolving Catalog omissions, positive associations are observed across the evaluated domains; for example, the GSMArena omission correlation is 0.667 for Pearson, Spearman, and Kendall. In contrast, Blueprint-Iterative Refinement has a negative GSMArena additions correlation of -0.509 , and several rows remain

undefined because one rater assigned a constant score (Table 5.10). These results provide limited support for using the judge to rank instances when its scores vary, but they do not prove that LLM-as-a-judge evaluation is generally reliable.

Several aspects of the evaluation constrain generalization. The human study contains ten instances per transition and domain and is scored by the thesis author, so it is an exploratory expert evaluation rather than an inter-annotator reliability study. The automatic pipeline evaluation covers five domains, whereas the human evaluation and fine-tuning evaluation cover four; ice hockey is not part of the human comparison. In addition, the use of related LLM capabilities in both generation and judging can introduce shared preferences. Reference metrics also assess similarity to a particular lexicalization rather than all valid ways of expressing a triple set. Finally, SENLEN is intentionally a simple period-delimited structural diagnostic and is not a semantic evaluation measure. These limitations support combining metrics and human inspection rather than treating any one score as definitive.

6.4 End-to-end evaluation and future work

The separate experiments establish the behavior of the two stages, but they do not establish the quality of the complete system on raw data. The planned whole-system evaluation addresses this gap by evolving a domain-specific surface-realization program on the independent S_3 collection and evaluating it on the separately collected S_4 collection. It will measure whether errors or omissions in generated triples propagate into the final text, which cannot be inferred reliably from isolated triple F1 or WebNLG scores.

At present, the whole-system numerical results are pending and therefore no performance conclusion should be drawn from Table 5.35. Even after those results are available, their interpretation will require care: the S_3 and S_4 reference texts and triples are generated by the selected fine-tuned model rather than independently annotated by humans. The evaluation will consequently measure agreement with distilled targets, not fully independent factual correctness with respect to the original data source.

The next evaluation cycle should therefore combine the completed end-to-end test with a small independently annotated reference set, multiple human raters, and agreement analysis between raters. Using judges from distinct model families, calibrating them with contrastive error cases, and retaining instance-level explanations would reduce the effect of score saturation. Further surface-realization experiments should report all variants on the same domains and include direct comparisons of inference latency, cost, and program maintainability. Together, these steps would determine when the proposed pipeline offers a practical advantage over direct prompting, rather than only demonstrating that its components can be optimized separately.

Chapter 7

Summary

This thesis investigated an interpretable data-to-text pipeline for converting heterogeneous structured data into natural-language text. Rather than treating generation as a single opaque transformation, the proposed approach separates the task into meaning-representation construction and surface realization:

$$d \longrightarrow T \longrightarrow y,$$

where d is a raw structured instance, T is a set of semantic triples, and y is the final realization. This decomposition exposes both the selected content and the procedure that expresses it. It therefore provides artifacts that can be inspected, evaluated, and modified independently: normalized triples at the meaning-representation stage and an executable Python program at the surface-realization stage.

7.1 Contributions and findings

Three LLM-based pipelines were developed for constructing semantic triples from raw structured data. Text-First Extraction with Normalization first generates a reference text, extracts triples independently for each instance, and then normalizes predicates after extraction. Blueprint-Iterative Refinement creates and validates a domain-specific extraction specification before batch extraction. Evolving Catalog incrementally constructs a predicate catalog and uses it to guide later extraction. These methods represent different trade-offs between coverage, throughput, predicate consistency, and human auditability.

The experiments identify Text-First Extraction with Normalization as the strongest of the evaluated pipeline methods. It obtains an automatic overall text-to-triples score of 4.996 and a human-evaluation overall score of 4.913. Evolving Catalog ranks second, while Blueprint-Iterative Refinement ranks third. The broader predicate vocabulary observed for Text-First Extraction with Normalization is compatible with its stronger coverage of varied facts, whereas the catalog and blueprint approaches make vocabulary control more explicit and auditable. Their weaker omission results show that restrictive, early vocabulary decisions can fail to cover facts that arise later in a domain. The results consequently favor post-hoc normalization when semantic coverage is the primary objective, while the other methods remain useful where explicit control of the extraction specification is more important.

The thesis also examined whether the multi-stage conversion process could be distilled into a domain-specialized model. QLoRA fine-tuning substantially improves mean triple F1 over the base Gemma model, from 0.0205 to a best reported value of 0.2340. The improvement is not uniform across domains: GSMArena and Wikidata benefit much more than OpenWeather and OWID, and

no single fine-tuning configuration leads on every text and triple metric. The fine-tuned model is therefore a practical way to reduce repeated inference-time pipeline calls, but it should be treated as a distillation of pipeline-produced targets rather than as an independently validated semantic model.

For surface realization, the thesis adapted the AlphaEvolve idea through OpenEvolve to search for Python programs that verbalize triples. LLMs propose program mutations, and the evolutionary loop executes and evaluates each candidate before retaining useful programs. This produces a reusable, inspectable realization artifact that can run on unseen triple sets without an inference-time LLM call. The experiments further show that the selection objective matters. BLEU-oriented fitness and LLM-as-a-judge fitness favor different properties: the former produces stronger reference-oriented results in some cases, whereas the latter produces stronger judge scores and lower detected additions and omissions. Removing inspiration programs improves BLEURT in the direct paired comparison, but does not consistently improve BLEU.

The baseline comparison places this contribution in context. The prompted LLM has the strongest reported judge-oriented scores, grammaticality, and detected additions and omissions, while fine-tuned BART has the strongest reference-based BLEU, BLEURT, and METEOR scores on the selected WebNLG domains. The evolved programs do not currently dominate these baselines on every quality metric. Their principal contribution is instead transparency: the chosen realization logic is represented by ordinary code that can be inspected, debugged, and adapted by a developer. This makes evolutionary program construction most appropriate when a reusable and controllable generator is required, rather than when the only objective is the highest benchmark score.

7.2 Evaluation conclusions

The evaluation confirms the importance of combining automatic measures with human assessment. The shared data-to-text stage receives near-ceiling automatic scores, with average summary and faithfulness results of 4.996 and 4.974. However, human evaluation reports lower means of 4.50 and 4.20, respectively, and identifies weaker faithfulness for GSMarena and OpenWeather. The difference shows that a high judge score should not be interpreted as conclusive evidence of factual fidelity.

The correlation analysis leads to the same conclusion. Data-to-text correlations cannot be calculated because the LLM judge assigns a constant score of 5 to all evaluated instances. For text-to-triples, several omission ratings show positive associations between the LLM and human evaluator, but other rows are undefined because of constant scores and the Blueprint GSMarena additions result is negatively correlated. LLM-as-a-judge evaluation is therefore useful as a scalable diagnostic when it differentiates cases, but it does not replace human assessment or establish universal evaluation reliability. The study’s single-author, small-sample human evaluation should be regarded as an exploratory validation rather than a complete annotation study.

7.3 Limitations and future work

The separate evaluations establish properties of the component methods, but the complete raw-data-to-text system has not yet been evaluated empirically. The planned S_3/S_4 evaluation will test whether errors introduced during meaning-representation construction propagate into the final realization on independently collected instances. Until these results are available, the thesis does not claim measured end-to-end performance. Moreover, this evaluation uses references and triples

generated by the selected fine-tuned model, so it will measure agreement with distilled targets rather than fully independent factual correctness with respect to the raw source data.

Future work should complete this end-to-end experiment and supplement it with independently annotated examples, multiple human raters, and inter-annotator agreement analysis. More diverse and calibrated judge models would help reduce score saturation. Further comparisons should evaluate all evolutionary variants on the same domains and measure inference latency, compute cost, maintenance effort, and the value of program inspectability in realistic use cases.

In conclusion, the thesis establishes a practical research framework for interpretable data-to-text generation. It demonstrates that LLMs can be used both to construct explicit semantic representations and to evolve executable surface-realization programs. The strongest current evidence favors flexible text-first extraction with post-hoc normalization for meaning representation, while positioning evolved programs as transparent and reusable alternatives to direct prompting. The remaining end-to-end evaluation will determine how these component-level advantages transfer to complete data-to-text generation from raw structured data.

Bibliography

- [1] Ron Artstein and Massimo Poesio. Survey article: Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4):555–596, 2008.
- [2] Satanjeev Banerjee and Alon Lavie. Meteor: An automatic metric for mt evaluation with improved correlation with human judgments. <https://aclanthology.org/W05-0909/>, 2005.
- [3] Thiago Castro Ferreira, Chris van der Lee, Emiel van Miltenburg, and Emiel Krahmer. Neural data-to-text generation: A comparison between pipeline and end-to-end architectures. <https://arxiv.org/abs/1908.09022>, 2019.
- [4] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.
- [5] Richard Cyganiak, David Wood, and Markus Lanthaler. Rdf 1.1 concepts and abstract syntax. <https://www.w3.org/TR/rdf11-concepts/>, 2014.
- [6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. <https://arxiv.org/abs/2305.14314>, 2023.
- [7] Bhuwan Dhingra, Manaal Faruqui, Ankur Parikh, Ming-Wei Chang, Dipanjan Das, and William Cohen. Handling divergent reference texts when evaluating table-to-text generation. <https://arxiv.org/abs/1906.01081>, 2019.
- [8] Ondrej Dušek and Zdenek Kasner. Evaluating semantic accuracy of data-to-text generation with natural language inference. <https://arxiv.org/abs/2011.10819>, 2020.
- [9] Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. The WebNLG challenge: Generating text from RDF data. <https://aclanthology.org/W17-3518/>, 2017.
- [10] Albert Gatt and Emiel Krahmer. Survey of the state of the art in natural language generation: Core tasks, applications and evaluation. <https://arxiv.org/abs/1703.09902>, 2018.
- [11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. <https://arxiv.org/abs/2106.09685>, 2021.
- [12] Zdeněk Kasner and Ondřej Dušek. Beyond traditional benchmarks: Analyzing behaviors of open llms on data-to-text generation. <https://arxiv.org/abs/2401.10186>, 2024.
- [13] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. <https://arxiv.org/abs/1412.6980>, 2014.
- [14] Klaus Krippendorff. Reliability in content analysis: Some common misconceptions and recommendations. *Human Communication Research*, 30(3):411–433, 2004.
- [15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. <https://arxiv.org/abs/2309.06180>, 2023.

- [16] Mateusz Lango and Ondrej Dusek. LLM agents implement an NLG system from scratch: Building interpretable rule-based RDF-to-text generators. <https://aclanthology.org/2025.emnlp-industry.142/>, 2025.
- [17] Sourab Mangrulkar, Sylvain Gugger, Lysak Lysak, et al. PEFT: Parameter-efficient fine-tuning of billion-scale models on low-resource hardware. <https://github.com/huggingface/peft>, 2022.
- [18] Amit Moryossef, Yoav Goldberg, and Ido Dagan. Step-by-step: Separating planning from realization in neural data-to-text generation. <https://arxiv.org/abs/1904.03396>, 2019.
- [19] Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. <https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/AlphaEvolve.pdf>, 2025.
- [20] Jekaterina Novikova, Ondrej Dušek, Amanda Cercas Curry, and Verena Rieser. Why we need new evaluation metrics for NLG. <https://arxiv.org/abs/1707.06875>, 2017.
- [21] Chinonso Cynthia Osuji, Thiago Castro Ferreira, and Brian Davis. A systematic review of data-to-text nlg. <https://arxiv.org/abs/2402.08496>, 2024.
- [22] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Ray, et al. Training language models to follow instructions with human feedback. <https://arxiv.org/abs/2203.02155>, 2022.
- [23] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: A method for automatic evaluation of machine translation. <https://aclanthology.org/P02-1040/>, 2002.
- [24] Ehud Reiter and Robert Dale. Building applied natural language generation systems. https://www.researchgate.net/publication/2839784_Building_applied_natural_language_generation_systems, 1997.
- [25] Thibault Sellam, Dipanjan Das, and Ankur Parikh. BLEURT: Learning robust metrics for text generation. <https://arxiv.org/abs/2004.04696>, 2020.
- [26] Asankhaya Sharma. Openevolve: An open-source evolutionary coding agent. <https://github.com/algorithmicsuperintelligence/openevolve>, 2025.
- [27] Gemma Team et al. Gemma: Open models based on gemini research and technology. <https://arxiv.org/abs/2403.08295>, 2024.
- [28] Gemma Team et al. Gemma 4 technical report. <https://arxiv.org/abs/2607.02770>, 2026.
- [29] Chris van der Lee, Emiel Krahmer, and Sander Wubben. Automated learning of templates for data-to-text generation: Comparing rule-based, statistical and neural methods. <https://aclanthology.org/W18-6504/>, 2018.
- [30] Huanting Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. Ai agentic programming: A survey of techniques, challenges, and opportunities. <https://arxiv.org/abs/2508.11126>, 2025.
- [31] Zhuoer Wang, Marcus Collins, Nikhita Vedula, Simone Filice, Shervin Malmasi, and Oleg Rokhlenko. Faithful low-resource data-to-text generation through cycle training. <https://aclanthology.org/2023.acl-long.160/>, 2023.
- [32] Jędrzej Warczyński, Mateusz Lango, and Ondrej Dusek. Leveraging large language models for building interpretable rule-based data-to-text systems. <https://arxiv.org/abs/2502.20609>, 2024.

- [33] Sam Wiseman, Stuart M. Shieber, and Alexander M. Rush. Challenges in data-to-document generation. <https://arxiv.org/abs/1707.08052>, 2017.
- [34] Jiannan Xiang, Zhengzhong Liu, Yucheng Zhou, Eric Xing, and Zhiting Hu. ASDOT: Any-shot data-to-text generation with pretrained language models. <https://aclanthology.org/2022.findings-emnlp.136/>, 2022.
- [35] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert. <https://arxiv.org/abs/1904.09675>, 2020.

Appendix A

Prompts

A.1 Evolution prompts for surface realization

The following prompts are the experiment-specific templates used to evolve the surface-realization program. Values in braces are filled at runtime by OpenEvolve. The templates are shown separately because the final user message is assembled from the mutation template, the current program, evaluation metrics, evolution history, and optional artifacts.

A.1.1 Evolution system prompt

You are an expert data engineer specializing in converting data to text. You will be given the current Python program with its score and your task will be to evolve the current Python program to improve the score. It is a program that converts a list of triples into natural language text. The given triples will be associated with the topic: {domain}. The program must implement a function called 'predict' that accepts a list of triples and generates a coherent, contextually relevant sentence that accurately represents the information contained in the triples. Ensure that the generated text is fluent, grammatically correct, and maintains the meaning of the original data. There can be multiple triples given that make up a complex sentence. Converting all those triples into one sentence will award greater score. An example of a complex sentence:

Triples:

(Antwerp | cityServed | Antwerp International Airport)

(Belgium | country | Antwerp)

(City of Brussels | capital | Belgium)

Example sentence:

"Antwerp International Airport serves the city of Antwerp which is in Belgium, where the capital is Brussels."

The 'predict' function returns that sentence as a string. Below is the list of all possible predicates that can be given as an input. With the given predicates and example triples implement the 'predict' function so it can process all the possible predicates:

{triples}

A.1.2 Evolution user prompt

The custom user template for diff-based evolution is:

```
# Current Program Information
{metrics}

{artifacts}

{evolution_history}

# Current Program
““{language}
{current_program}
““

# Task
Suggest an improvement to the current program that will improve the combined_score.
Different solutions with similar combined_score score but different ideas are valuable.
Creating one complex sentence from all the given triples will award greater score as well.

You MUST use the exact SEARCH/REPLACE diff format shown below to indicate changes:

<<<<<< SEARCH
# Original code to find and replace (must match exactly, indentations and endlines as well
    )
=====
# New replacement code
>>>>>> REPLACE

Example of a valid diff format:
<<<<<< SEARCH
    best_x = 0
    best_y = 0
=====
    # Initialize with a random point
    best_x = np.random.uniform(bounds[0], bounds[1])
    best_y = np.random.uniform(bounds[0], bounds[1])
>>>>>> REPLACE
```

The SEARCH section must exactly match the code in the current program, even with it's indentations and endlines.

Do not provide explanations, only the SEARCH/REPLACE section.

IMPORTANT: Do not rewrite the entire program - focus on targeted improvements.

The placeholders are expanded as follows: `metrics` contains the parent's recorded metrics, `artifacts` contains the optional artifact message shown in Appendix A.1.4, `evolution_history` contains previous attempts and selected programs, and `current_program` contains the parent source code. In the reported run, the top-program and inspiration subsections were disabled, but the inspiration slot was enabled separately.

A.1.3 Evolution-history components

The user prompt can contain the following runtime-generated history components. They are included here to make the composition of the complete user message explicit.

```
{inspirations_section}

### Attempt {attempt_number}
- Changes: {changes}
- Metrics: {performance}
- Outcome: {outcome}

### Program {program_number} (Score: {score})
““{language}
{program_snippet}
““
Key features: {key_features}

## Inspiration Programs

This is a different program that might have a different approach on how to solve the
    problem. Use it as an inspiration. Do not copy the whole program:

{inspiration_programs}

### Inspiration {program_number} (Score: {score})
““{language}
{program_snippet}
““
```

A.1.4 Artifact message

Artifacts returned by the evaluator are wrapped before being inserted into the user prompt. The wrapper has the following form:

```
## Evaluator Artifacts

### {artifact_key}
““
{artifact_content}
““
```

The artifact content is truncated to the configured maximum size and passed through the artifact security filter. For example, a low-scoring LLM-judge artifact has the following structure:

```
The program got scored by the judge with the score {rating}. The input triples were:
{subject} | {predicate} | {object}
```

```
The generated text was:
{generated_text}
```

```
The judge review is:
{review}
```

Based on the review try to understand why the program might have performed poorly and improve the program accordingly.

A.1.5 Repair prompts

When no valid diff is extracted, the following repair messages are used. The second user message is also used when a syntactically valid diff does not apply any change to the parent source.

SYSTEM:

You are an expert in fixing your colleagues code. You know that the code should be in format:

<<<<<< SEARCH

Original code to find and replace (must match exactly, indentations and endlines as well)

=====

New replacement code

>>>>>> REPLACE

Provided the incorrect format of SEARCH/REPLACE fix it to a correct format.

USER:

The original code was:

{parent_code}

An incorrect diff format was detected in this change:

{llm_response}

Please fix it to the correct format.

SYSTEM:

You are an expert in fixing your colleagues code. You know that in order for the SEARCH/REPLACE diff to be correct, the SEARCH section must EXACTLY match a part of the original code. That includes indentations and endlines as well. Every character matters. The provided code in the SEARCH section DOES NOT match exactly the original code. Your task is to modify the SEARCH section so that it matches EXACTLY the searched part of the original code. Make sure to keep the REPLACE section unchanged.

USER:

The provided diff did not apply any changes to the original code:

{llm_response}

Given the original code please modify the SEARCH section to ensure it matches exactly the searched part of the original code:

{parent_code}

The spelling in these blocks is preserved from the implementation because the appendix documents the actual prompts sent to the model.

A.2 LLM-as-a-judge prompt

A.2.1 Meaning-representation pipeline prompts

The meaning-representation pipeline evaluation uses four independent criteria, split across two transitions. The data-to-text transition uses summary and faithfulness; the text-to-triples transition uses additions and omissions. Each criterion is scored with a separate prompt on an integer

scale from 1 (worst) to 5 (best). The judge is instructed to assess only the relationship between the supplied source and output, without using external knowledge:

Data to text The source is the original structured data instance and the output is the generated natural-language reference text.

Text to triples The source is the generated reference text and the output is the generated semantic triples.

The task-specific part of the system prompt is instantiated as follows:

You are a strict judge evaluating a {conversion} conversion.

You will see: (1) {source_label}, and (2) {output_label}.

{criterion_guideline}

Use only the provided source and output. Assign one integer score from 1 to 5.

Return ONLY JSON with this exact schema:

```
{"score": <integer 1-5>, "reason": "<one short sentence>"}
```

For the data-to-text transition, `conversion` is “data-to-text”, `source_label` is “the original structured data instance”, and `output_label` is “the natural-language text generated from it”. For the text-to-triples transition, the corresponding values are “text-to-triples”, “the natural-language reference text”, and “the semantic triples generated from it”. The current criterion guidelines are:

Summary

Criterion: Summary

Evaluate whether the natural-language text is a concise, coherent summary of the original structured data instance. It should avoid repeating the same concept with different words, be significantly shorter than detailed or repetitive structured data when possible, preserve the core message, and function as a summary rather than a rewritten essay or a disjointed list of facts.

Score 5: The natural-language text is a clear and coherent summary. It preserves the original structured data’s core message without redundant restatement and does not read like an essay or a disconnected list.

Score 4: The natural-language text is a good summary and preserves the core message, but has minor redundancy, slight unnecessary length, or a small organizational weakness.

Score 3: The natural-language text is recognizably a summary, but is noticeably repetitive, too verbose, list-like, or weakly organized; its main message remains usable.

Score 2: The natural-language text is a poor summary: it substantially repeats or rewrites the original structured data, is very verbose or disjointed, and represents the core message only partly.

Score 1: The natural-language text does not function as a summary. It is mostly irrelevant, repetitive, disjointed, or fails to communicate the original structured data’s core message.

Additions

Criterion: Additions

Evaluate whether the semantic triples introduce facts, relationships, values, qualifiers, or conclusions that are not supported by the natural-language reference text. Penalize unsupported additions, hallucinations, contradictions, and material distortions. Do not penalize information that is merely omitted; evaluate omissions under Omissions.

Score 5: The semantic triples contain no unsupported additions. Every represented fact, relationship, value, qualifier, and conclusion is supported by the natural-language reference text, with no hallucination or material distortion.

Score 4: The semantic triples contain at most one or two minor unsupported additions or imprecisions, but no material fact is fabricated and the text’s meaning is preserved.

Score 3: The semantic triples contain several minor unsupported additions or one material unsupported fact or distortion, but most represented information is grounded in the text.

Score 2: The semantic triples contain multiple material unsupported facts, relationships, values, or conclusions, or substantially misrepresent the reference text.

Score 1: The semantic triples are mostly unsupported, contradictory, or materially different from the natural-language reference text.

Faithfulness

Criterion: Faithfulness

Evaluate whether every claim in the natural-language text is directly supported by the original structured data instance and whether entities, relationships, values, qualifiers, and their meaning are represented accurately. Penalize hallucinations, fabrications, contradictions, and misinterpretations. Do not penalize information that is merely omitted.

Score 5: Every claim is grounded in the original structured data instance, and all represented facts preserve the source’s meaning. There are no hallucinations, contradictions, or material distortions.

Score 4: The natural-language text is fully usable and essentially faithful, with at most a minor imprecision that does not change the meaning and no material unsupported claim.

Score 3: Most claims are grounded, but there is one material error or distortion, or several minor unsupported or inaccurate claims.

Score 2: There are multiple material errors, hallucinations, contradictions, or a substantial misinterpretation of the original structured data.

Score 1: The natural-language text is mostly ungrounded or contradicts the original structured data, so it cannot be considered a reliable representation.

Omissions

Criterion: Omissions

Evaluate what the semantic triples leave out from the output text. Lower the score when omissions remove critical context, change the meaning, hide a major conclusion or constraint, or create a misleading imbalance or bias.

Score 5: Only redundant material is omitted. No omission changes the meaning, hides important context, or introduces a meaningful bias.

Score 4: One or a few minor omissions exist, but the meaning and conclusions remain intact and the selection of included information is not meaningfully biased.

Score 3: An important but non-central detail, or enough supporting context to create a mild imbalance, is omitted; the overall message remains understandable.

Score 2: A critical context element, major conclusion, important constraint, or representative point is omitted, changing the meaning or creating clear bias.

Score 1: Extensive omissions distort the core message or make the output triples misleading or one-sided; most important source information is absent.

A.2.2 Surface-realization judge prompts

For every non-empty generated realization, the evaluator sends the input triples and the generated text to the configured LLM judge. The reference texts are not included in this request. The prompt used by the evaluator is shown below; **source** is the formatted list of triples and **target** is the generated text.

###Instruction###

Please act as an impartial and helpful evaluator for natural language generation (NLG), and the audience is an expert in the field.

Your task is to evaluate the quality of text similarity strictly based on the given evaluation criterion.

Begin the evaluation by providing your analysis concisely and accurately, and then on the next line, start with "Rating:" followed by your rating on a Likert scale from 1 to 5 (higher means better).

You MUST keep to the strict boundaries of the evaluation criterion and focus solely on the issues and errors involved; otherwise, you will be penalized.

Make sure you read and understand these instructions, as well as the following evaluation criterion and example content, carefully.

###Evaluation Criterion###

Accuracy and number of sentences: The generated text must accurately reference the triples. There cannot be any extra information that was not present in the triples. If possible there must be just one complex sentence instead of multiple sentences.

###Data###

The triples in the form (subject, predicate, object):
{source}

The generated text:
{target}

###Your Evaluation###

The response is parsed as a review followed by a rating from 1 to 5. The ratings are averaged and divided by 5 before they are used as the evolutionary fitness.

A.2.3 Grammaticality prompt

The following criterion is used for the binary grammaticality score. The judge returns 1 for a grammatically correct realization and 0 otherwise.

Grammatical correctness: You should assess the grammatical correctness of the resulting text. Do not take any other factors into account. Do not make assumptions or consider external knowledge not present in the provided context. Identify only errors relating to the grammaticality of the text. Do not consider aspects such as fluency, omissions or hallucinations.

Return 1 if the text is grammatically correct and 0 if it is not. The rating must be the integer 0 or 1.

A.2.4 Omissions prompt

The omissions score measures whether any input triples were not verbalized. A score of 0 indicates that no omission was detected, while 1 indicates that at least one omission was detected.

Omissions: You should assess the omissions in the resulting text; in other words, check whether any of the input triples were not verbalised. You can perform the task by iterating over the input triples and checking if each is present in the output. Do not take any other factors into account. Do not make assumptions or consider external knowledge not present in the provided context. Identify only omissions. Do not consider grammaticality, fluency or the addition of new facts.

Return 0 if there are no omissions and 1 if there are. The rating must be the integer 0 or 1.

A.2.5 Additions prompt

The additions score measures whether the realization introduces facts that are not supported by the input triples. A score of 0 indicates that no addition was detected, while 1 indicates that at least one addition was detected.

Additions: You should assess the addition of new facts in the resulting text which were not present in the input triples. Carefully read the text and check whether the facts mentioned are present in the input triples. Do not take any other factors into account. Do not make assumptions or consider external knowledge not present in the provided context. Identify only additions. Do not consider grammaticality, fluency or omissions.

Return 0 if there are no additions and 1 if there are. The rating must be the integer 0 or 1.

A.2.6 Zero-shot prompt

The zero-shot prompt is used to convert the input triples into a natural language realization without any additional context. The prompt is shown below;

You are an expert data-to-text generator. Verbalize the given RDF triples into a single fluent, grammatically correct English sentence that conveys every fact and adds no extra information.

Convert these RDF triples (subject | predicate | object) into one natural-language sentence.

Triples:
{triples}

Output only the sentence.

A.3 Text-First Extraction with Normalization

This section contains the complete text of all system prompts used in the Text-First Extraction with Normalization method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: {"text":"..."} . Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

Triples-From-Text System Prompt (P_{tr})

You extract semantic triples from natural language text. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...", "predicate":"...", "object":"..."}]}. Rules: preserve the full meaning of the input text, avoid duplicate triples, and keep predicates in lower_snake_case when possible.

Candidate-Pairs System Prompt

You find candidate predicate pairs that may be semantically equivalent. Return ONLY JSON with schema: {"pairs":[{"predicate_a":"...", "predicate_b":"..."}]}. Rules: include only pairs with high likelihood of strict semantic equivalence; do not include pairs that are merely related; do not include duplicates or self-pairs.

Strict-Equivalence System Prompt

You decide if two predicates are semantically equivalent in meaning. Return ONLY JSON with schema: {"equivalent": true|false, "confidence": 0.0-1.0, "reason": "..."} . Be strict: equivalent only if they can be safely merged in a knowledge graph.

A.4 Blueprint-Iterative Refinement

This section contains the complete text of all system prompts used in the Blueprint-Iterative Refinement method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: `{"text":"..."}`. Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

Blueprint-Creation System Prompt ($P_{\text{blueprint}}$)

You design extraction rules for a domain. Return ONLY JSON with schema: `{"predicates":["..."], "example_triples":[{"subject":"...", "predicate":"...", "object":"..."}], "example_text":"...", "guidelines":["..."]}`. Rules: include precise and reusable predicates, provide concrete example triples with strict formatting, provide an example text matching those triples, and give practical transformation guidelines.

Blueprint-Feasibility System Prompt (P_{feas})

You evaluate whether given extraction rules are sufficient to extract triples from a text. Return ONLY JSON with schema: `{"possible":true|false, "reason":"...", "rule_gaps":["..."]}`.

Blueprint-Revision System Prompt (P_{rev})

You revise extraction rules so triples can be produced from a text. Return ONLY JSON with schema: `{"predicates":["..."], "example_triples":[{"subject":"...", "predicate":"...", "object":"..."}], "example_text":"...", "guidelines":["..."]}`. Keep existing high-quality rules, but add or adjust what is necessary to cover the failing case.

Triples-From-Text-With-Blueprint System Prompt ($P_{\text{tr\&R}}$)

You extract semantic triples from natural language text using provided rules. Return ONLY JSON with this exact schema: `{"triples":[{"subject":"...", "predicate":"...", "object":"..."}]}`. Follow rule guidelines and mimic the formatting style from example triples. Prefer predicates from the rule predicate list whenever possible.

A.5 Evolving Catalog

This section contains the complete text of all system prompts used in the Evolving Catalog method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: `{"text":"..."}`. Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

First-Instance System Prompt (P_{first})

You extract semantic triples from text. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...", "predicate":"...", "object":"..."}]}. Use concise predicate labels in lower_snake_case when possible and avoid duplicates.

Catalog-Guided System Prompt (P_{cat})

You extract semantic triples from text using a predicate catalog with examples. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...", "predicate":"...", "object":"..."}]}. Use only predicates from the provided catalog when possible. Introduce a new predicate only when none of the catalog predicates can describe the fact.

Appendix B

Prompts for Fine-tuned Model

System Prompt for Joint Text and Triple Generation

You convert one structured data instance into concise natural language and a corresponding set of RDF semantic triples.

Return ONLY JSON with schema: `{"text":"...", "triples":[{"subject":"...", "predicate":"...", "object":"..."}]}`.

Capture the most important information, trends, extremes, and notable conditions.

Use concise predicate labels in lower_snake_case when possible and avoid duplicates.

If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

User Message Template

Create a concise natural-language summary of this ONE data instance and extract its semantic triples.

`instance_context={serialized JSON instance}`

Return JSON only.

Assistant Message (Expected Output)

```
{"text": "<generated natural language text>", "triples": [{"subject": "...", "predicate": "...", "object": "..."}, ...]}
```

Appendix C

Initial Program

The following program is the complete initial source used to start surface realization evolution. The code outside the evolution block defines the data type and the public interface; the marked block is the part that OpenEvolve is allowed to modify.

```
from dataclasses import dataclass

@dataclass
class Triple:
    subject: str
    predicate: str
    object: str

# EVOLVE-BLOCK-START

def predict(triples: list[Triple]) -> str:
    return f"The {triples[0].predicate} of {triples[0].subject} is {triples[0].object}."

# EVOLVE-BLOCK-END
```



© 2026 Dominik Maćkowiak

Poznan University of Technology
Faculty of Computing
Institute of Computing Science

Typeset using L^AT_EX in Computer Modern.

BibT_EX:

```
@mastersthesis{ key,  
  author = "Dominik Maćkowiak",  
  title = "{Interpretable methods for data-to-text generation}",  
  school = "Poznan University of Technology",  
  address = "Pozna{\'}n, Poland",  
  year = "2026",  
}
```